

Dawwarha Frontend Implementation Plan

A practical execution guide for the Angular + Tailwind frontend

NTI · MEAN Stack Graduation Project · Version 1.0

Prepared for the development team, QA, and product review

- [Dawwarha — Frontend Implementation Plan](#)
- [00 — Purpose & Source-of-Truth Hierarchy](#)
- [01 — Executive Summary](#)
- [02 — Architecture](#)
- [03 — Technology Stack](#)
- [04 — Project Structure](#)
- [05 — Design System Integration](#)
- [06 — Frontend Domain Map](#)
- [07 — Cross-Engineer Contracts](#)
- [08 — Engineer 1 — Shared Foundation](#)
- [09 — Engineer 2 — Supply](#)
- [10 — Engineer 3 — Demand & Matching](#)
- [11 — Engineer 4 — Transfer, Trust & Impact](#)
- [12 — Authentication](#)
- [13 — Authorization / RBAC](#)
- [14 — API Contract Integration](#)
- [15 — Data Models / TypeScript Types](#)
- [16 — Forms & Validation](#)
- [17 — Error Handling](#)
- [18 — Loading States](#)
- [19 — Empty States](#)
- [20 — Responsive Implementation](#)
- [21 — RTL / LTR](#)
- [22 — Accessibility](#)
- [23 — State Management](#)
- [24 — Routing Plan](#)
- [25 — Page Inventory](#)
- [26 — Shared Component Inventory](#)
- [27 — Testing Strategy](#)
- [28 — Mock Data / Development Strategy](#)
- [29 — Git Workflow](#)
- [30 — Integration Plan](#)
- [31 — Dependency Graph](#)
- [32 — Critical Path](#)
- [33 — Risks & Common Failure Points](#)
- [34 — Performance](#)
- [35 — Security](#)
- [36 — SEO / Public Experience](#)
- [37 — Deployment](#)
- [38 — Documentation](#)
- [39 — Definition of Done](#)
- [40 — Complete Task Checklist](#)
- [41 — Graduation Demo Flow](#)
- [42 — Evaluator Experience](#)
- [43 — Final Architecture Summary](#)

Dawwarha — Frontend Implementation Plan

A practical execution guide for the Angular, TypeScript, and Tailwind CSS frontend.

Version 1.0 · Companion document to *Dawwarha — Backend Implementation Plan v1.0* Audience: 4 beginner-level frontend engineers, executing from an empty Angular repository to a graduation-ready demo.

Team:

Engineer	Name	Domain
Engineer 1	رحمة محمد محمد أحمد	Identity + Organizations + Shared Application Foundation
Engineer 2	ادهم محمد حسن عبد السمیع	Supply — Categories + Resources
Engineer 3	دینا عبدالله علیان محمد	Demand & Matching — Requests + Matches
Engineer 4	عبدالستار محمد عبدالستار مطاوع	Transfer + Trust + Impact + Notifications

Contents

0. Purpose & Source-of-Truth Hierarchy
 1. Executive Summary
 2. Architecture
 3. Technology Stack
 4. Project Structure
 5. Design System Integration
 6. Frontend Domain Map
 7. Cross-Engineer Contracts
 8. Engineer 1 — Shared Foundation
 9. Engineer 2 — Supply
 10. Engineer 3 — Demand & Matching
 11. Engineer 4 — Transfer, Trust & Impact
 12. Authentication
 13. Authorization / RBAC
 14. API Contract Integration
 15. Data Models / TypeScript Types
 16. Forms & Validation
 17. Error Handling
 18. Loading States
 19. Empty States
 20. Responsive Implementation
 21. RTL / LTR
 22. Accessibility
 23. State Management
 24. Routing Plan
 25. Page Inventory
 26. Shared Component Inventory
 27. Testing Strategy
 28. Mock Data / Development Strategy
 29. Git Workflow
 30. Integration Plan
 31. Dependency Graph
 32. Critical Path
 33. Risks & Common Failure Points
 34. Performance
 35. Security
 36. SEO / Public Experience
 37. Deployment
 38. Documentation
 39. Definition of Done
 40. Complete Task Checklist
 41. Graduation Demo Flow
 42. Evaluator Experience
 43. Final Architecture Summary
-

00 — Purpose & Source-of-Truth Hierarchy

This document is the frontend equivalent of *Dawwarha — Backend Implementation Plan v1.0*. It follows the same philosophy: no page is “built” until it is validated, tested, accessible, responsive, RTL-correct, and integrated — not merely rendered. It is written for four beginner Angular engineers and assumes no prior familiarity with the surrounding documents; every non-trivial concept is explained inline.

This plan does **not** redefine product functionality, database schema, API semantics, lifecycle rules, matching weights, or authorization rules. It explains how Angular consumes and visually represents those contracts. The source-of-truth hierarchy is:

Concern	Authoritative document
Product functionality, roles, workflows, lifecycle states	Dawwarha-Product-Brief.pdf
Database schema, indexes, embedding/referencing	Dawwarha – MongoDB Implementation Plan v1.0
API contract (endpoints, request/response shape, status codes)	Dawwarha – Backend Implementation Plan v1.0
Visual/UI system (tokens, components, patterns)	DESIGN.md (primary) and DAWWARHA-DESIGN-SYSTEM.md (equivalent, more granular restatement) — the two are functionally aligned; where a wording difference exists, DESIGN.md’s Section numbering is used for cross-references in this plan since it is the more recently structured of the two
Frontend execution	this document

Cross-Engineer Contract Gap protocol: if a frontend task needs information the Backend Plan does not specify (e.g., an exact field name, a missing error code for a specific edge case), the correct action is to flag it explicitly as a **Cross-Engineer Contract Gap** in the PR description and in Section 07 of this document, and resolve it with the backend engineer who owns that domain — never to invent an endpoint, field, or status value silently. Every such gap found during execution must be appended to Section 07’s table before the affected task is marked done.

No invented API endpoints, lifecycle states, or roles appear anywhere in this document. Every endpoint referenced in Section 14 is copied verbatim from the Backend Plan’s Section 14 (API Specification) and Section 20 of the Product Brief. Every lifecycle state referenced anywhere in this document is copied verbatim from Product Brief Section 11 and Backend Plan Section 06.

01 — Executive Summary

Dawwarha's frontend is an Angular 17+ single-page application that gives the platform's three user populations — individual providers/seekers, community organizations, and platform administrators — a structured way to publish resources and requests, review transparent rule-based matches, coordinate a two-sided handover, and see their contribution recorded as platform impact.

The frontend's job is narrower than it sounds: it does not decide *what* the product does — the Product Brief and Backend Plan already do that — it decides how those decisions are *shown*. Every screen must make two things obvious at a glance, per `DESIGN.md` Section 2: what the user can do here, and what happens when they do it. The flagship trust surface of the whole product — the Match Score with its five-signal breakdown — must never appear as an unexplained percentage; that single requirement shapes a large share of Engineer 3's work (Section 10).

The team preserves the same four-way domain ownership used on the backend (Identity, Supply, Demand & Matching, Transfer/Trust/Impact), because the two teams' domain boundaries already match one-to-one with the API surface each frontend engineer consumes — reusing the split minimizes cross-team coordination overhead.

Guiding principle, restated from the Backend Plan: **do not over-engineer the MVP**. No NgRx unless a specific, documented state-sharing problem cannot be solved by a plain injectable service; no micro-frontends; no custom design-system tooling beyond a Tailwind token layer. A well-structured, modular Angular application that four beginners can understand end-to-end is the actual deliverable — not an impressive-looking architecture no one on the team can maintain after graduation.

02 — Architecture



Presentation layer (pages/routes): owns layout composition and route-level data resolution only — no business logic, no direct HTTP calls.

Feature layer (domain components + feature services): each domain (Resources, Requests, Matches, Handovers, ...) owns its own typed service that wraps `HttpClient` calls to its slice of the API, and its own domain components that understand product concepts (a `ResourceCard` knows what “category” and “availability window” mean; [DESIGN.md Section 31](#)).

Shared UI layer (`shared/ui/`): primitive, product-agnostic components — a `ButtonComponent` would be equally at home in an unrelated product ([DESIGN.md Section 31](#)). Every domain component is built from these, never from hand-rolled markup that duplicates what a primitive already solves.

Core services (`core/`): things that exist exactly once for the whole app — `AuthService` (session state), `authInterceptor`, `errorInterceptor`, route guards, the global `ToastService`, and app-wide configuration (`environment.ts`).

API layer: one typed service per backend domain (`AuthApiService`, `ResourceApiService`, `RequestApiService`, `MatchApiService`, `HandoverApiService`, `OrganizationApiService`, `ReportApiService`, `NotificationApiService`, `AdminApiService`), each mapping 1:1 to a routes file in the Backend Plan’s Section 04 project structure — this mirroring is intentional and keeps “which engineer owns which frontend service” trivially derivable from “which engineer owns which backend routes file.”

State management: covered fully in Section 23 — local component state, small feature-level state held in injectable services with RxJS `BehaviorSubject`s, and a single global `AuthSession` — no `NgRx` for the MVP (Section 23 explains why).

Models: one `core/models/*.ts` file per shared contract (Section 07), imported by every feature that touches that entity — never redeclared per-feature.

Guards & interceptors: functional guards (`authGuard`, `roleGuard`, `orgVerifiedGuard`) and functional interceptors (`authInterceptor`, `errorInterceptor`), Angular’s modern (non-class-based) style, registered once in `app.config.ts`.

Error handling: normalized centrally by `errorInterceptor` into a single `ApiError` shape before any component ever sees it (Section 17).

Configuration: `environment.ts` / `environment.production.ts` hold `apiBaseUrl` only — no secrets, per [DESIGN.md / Backend Plan security sections](#); nothing sensitive is ever stored in frontend code.

03 — Technology Stack

Concern	Choice	Why
Framework	Angular 17+	Matches the MEAN brief; modern Angular (standalone components, no NgModules) is simpler for beginners than the classic NgModule boilerplate
Language	TypeScript (strict mode)	Type-safety against the shared contracts in Section 07; <code>strict: true</code> in <code>tsconfig.json</code> from day one — retrofitting strictness later is much harder
Styling	Tailwind CSS, tokens from <code>DESIGN.md</code> Section 32	Design system is already Tailwind-native; no separate CSS framework
Component style	Standalone components	No <code>NgModule</code> files to keep in sync; a beginner adds a component by creating one file, not two
Reactivity	Signals for local/UI state (<code>isLoading</code> , <code>isOpen</code> , form-adjacent state) · RxJS for HTTP streams and anything event-based (router events, <code>valueChanges</code>)	Signals are simpler to reason about for beginners than manual subscription management; RxJS remains the right tool for streams that genuinely are streams (HTTP, router)
Routing	Angular Router, route-level lazy loading via <code>loadComponent</code> / <code>loadChildren</code> per feature	Keeps the initial bundle small; each domain (<code>resources</code> , <code>requests</code> , <code>matches</code> , <code>admin</code> , ...) is its own lazy chunk
Guards	Functional guards (<code>CanActivateFn</code>)	The modern, non-class Angular API — less boilerplate, easier for a beginner to read top-to-bottom
Interceptors	Functional interceptors (<code>HttpInterceptorFn</code>)	Same rationale as guards
Forms	Angular Reactive Forms	Required by the Product Brief (Section 18/21) and by <code>DESIGN.md</code> Section 19's guided-form direction; template-driven forms are not used anywhere in this project, for consistency
HTTP	<code>HttpClient</code> (Angular's built-in client)	No <code>axios</code> / <code>fetch</code> wrapper needed — <code>HttpClient</code> plus interceptors covers every requirement here
State management	Plain injectable services + Signals/RxJS <code>BehaviorSubject</code> s; no NgRx	See Section 23 — the MVP's state needs do not justify the boilerplate and learning curve NgRx adds for four beginners
Testing	Jasmine + Karma (Angular CLI default) for unit/component tests	Zero extra setup; ships with <code>ng new</code>
Icons	Lucide (via <code>lucide-angular</code>)	<code>DESIGN.md</code> Section 11 — single icon family, no exceptions
Env config	<code>environment.ts</code> files, Angular CLI file replacement for prod builds	Standard Angular pattern, no extra tooling

Explicitly not used, mirroring the Backend Plan's Section 03 "not used" list and the Product Brief's Section 51 scope discipline: React, Vue, Redux, NgRx (unless a specific documented case in Section 23 proves otherwise), micro-frontends, GraphQL, server-side rendering (Angular Universal — not required by any FR/NFR), a component library beyond the project's own primitives (no Angular Material, no PrimeNG — `DESIGN.md` Section 36 explicitly forbids "generic Angular Material styling applied unmodified").

04 — Project Structure

```
frontend/
├── src/
│   ├── app/
│   │   ├── core/
│   │   │   ├── auth/
│   │   │   │   ├── auth.service.ts          # session state (Eng 1)
│   │   │   │   ├── auth-session.model.ts    # shared AuthSession contract (Eng 1)
│   │   │   │   └── guards/
│   │   │   │       ├── auth.guard.ts        # authGuard (Eng 1)
│   │   │   │       ├── role.guard.ts        # roleGuard(...roles) (Eng 1)
│   │   │   │       └── org-verified.guard.ts # orgVerifiedGuard (Eng 1)
│   │   │   ├── interceptors/
│   │   │   │   ├── auth.interceptor.ts      # attaches JWT (Eng 1)
│   │   │   │   └── error.interceptor.ts     # normalizes errors (Eng 1)
│   │   │   ├── services/
│   │   │   │   ├── toast.service.ts        # global toast queue (Eng 1)
│   │   │   │   └── api-base.service.ts     # shared HttpClient helpers (Eng 1)
│   │   │   └── models/
│   │   │       ├── user.model.ts
│   │   │       ├── organization.model.ts
│   │   │       ├── resource.model.ts
│   │   │       ├── request.model.ts
│   │   │       ├── category.model.ts
│   │   │       ├── match.model.ts
│   │   │       ├── handover.model.ts
│   │   │       ├── contribution.model.ts
│   │   │       ├── notification.model.ts
│   │   │       ├── report.model.ts
│   │   │       ├── api-error.model.ts
│   │   │       └── pagination.model.ts
│   │   └── config/
│   │       └── app.config.ts                # providers, interceptor/guard registration
│   ├── shared/
│   │   ├── ui/
│   │   │   # Primitive Components – DESIGN.md Section 12
│   │   │   ├── button/ input/ textarea/ select/ combobox/
│   │   │   ├── checkbox/ radio/ switch/ search/ file-upload/
│   │   │   ├── card/ badge/ avatar/ tooltip/ dropdown/
│   │   │   ├── dialog/ drawer/ tabs/ toast/ alert/
│   │   │   ├── skeleton/ spinner/ table/ pagination/
│   │   │   └── components/
│   │   │       # Domain Components – DESIGN.md Section 13
│   │   │       ├── resource-card/ request-card/ match-card/ match-score/
│   │   │       ├── lifecycle-timeline/ location-chip/ urgency-badge/
│   │   │       ├── verification-badge/ impact-card/ contribution-card/
│   │   │       ├── organization-card/ notification-item/ report-status/
│   │   │       └── directives/
│   │   ├── pipes/
│   │   │   # e.g. relative-date.pipe.ts, unit-quantity.pipe.ts
│   │   └── utils/
│   ├── layout/
│   │   ├── public-layout/
│   │   │   # top nav, footer – Eng 1
│   │   ├── app-layout/
│   │   │   # sidebar/bottom-tab shell – Eng 1
│   │   └── admin-layout/
│   │       # distinct admin shell – Eng 1
│   ├── features/
│   │   ├── auth/
│   │   │   # Eng 1 – login, register
│   │   ├── organizations/
│   │   │   # Eng 1
│   │   ├── profile/
│   │   │   # Eng 1
│   │   ├── categories/
│   │   │   # Eng 2 (admin CRUD only; public read lives inside resources/requests filters)
│   │   ├── resources/
│   │   │   # Eng 2
│   │   ├── requests/
│   │   │   # Eng 3
│   │   ├── matches/
│   │   │   # Eng 3
│   │   ├── handovers/
│   │   │   # Eng 4
│   │   ├── contributions/
│   │   │   # Eng 4
│   │   ├── notifications/
│   │   │   # Eng 4
│   │   ├── reports/
│   │   │   # Eng 4
│   │   └── admin/
│   │       # shared – sub-routes owned per domain (Section 06)
│   ├── design-tokens/
│   │   └── tailwind-tokens.ts
│   │       # single source Tailwind config imports from – DESIGN.md Section 32
│   ├── environments/
│   │   ├── environment.ts
│   │   └── environment.production.ts
│   ├── styles.css
│   │   # Tailwind entry + global resets
│   ├── tailwind.config.js
│   ├── angular.json
│   ├── tsconfig.json
│   └── package.json
```

Why this structure, folder by folder:

- `core/` exists once, is eagerly loaded, and contains everything that must exist before any feature route can safely render — auth state, guards, interceptors, shared models. A beginner rule of thumb: *if two unrelated features both need it, it goes in `core/`, not copy-pasted into both.*
- `shared/ui/` vs `shared/components/` mirrors DESIGN.md Section 31's Primitive vs. Domain split exactly — this is not a frontend-only decision, it is the design system's own architecture, so the folder names were chosen to match it 1:1.
- `layout/` is separate from `features/` because a layout shell is not a “page” — it is the frame every page in a given context (public / authenticated / admin) renders inside. DESIGN.md Section 23 defines three distinct navigation contexts; this plan gives each its own shell component for the same reason the Design System gives Admin its own visually distinct header.
- `features/` is organized one folder per domain, matching Section 06's ownership table exactly — an engineer never has to guess which folder a task belongs in.

05 — Design System Integration

DESIGN.md and DAWWARHA-DESIGN-SYSTEM.md are authoritative for every visual decision in this project (Section 00). No engineer invents color, spacing, radius, shadow, motion, or component behavior — Section 45/37 of those documents (“AI Coding Agent Rules” / “AI Implementation Rules”) apply equally to human engineers on this team.

Consumption model:

1. **Tokens → Tailwind.** All tokens (DESIGN.md Section 4/6/9/10/11) are defined once in design-tokens/tailwind-tokens.ts and imported into tailwind.config.js's theme extension. Component markup uses semantic utility classes only (bg-primary, rounded-card, text-danger) — never a raw hex or an arbitrary bracket value (bg-[#164E41]) is a Design Anti-Pattern per DESIGN.md Section 36 and is flagged in every code review).
2. **Primitives first.** Before writing any new component, an engineer checks shared/ui/ for an existing primitive covering the need (Button, Input, Card, Badge, Dialog, Toast, Skeleton, ...). DESIGN.md Section 12 documents each primitive's full contract (Purpose, Anatomy, Variants, Sizes, States, Responsive, Accessibility, Do/Don't) — that section is the primitive's actual specification; this plan's tasks reference it by name rather than repeating it.
3. **Domain components before hand-rolled markup.** ResourceCard, RequestCard, MatchCard, MatchScore, LifecycleTimeline, LocationChip, UrgencyBadge, VerificationBadge, ImpactCard, ContributionCard, OrganizationCard, NotificationItem, ReportStatus are specified in full in DESIGN.md Sections 13–19 — Engineers build these once, in shared/components/, and every page composes them. No engineer re-implements a Resource Card's anatomy inline on a page component.
4. **No engineer invents their own UI.** If a screen needs a pattern not covered by an existing primitive/domain component/page template, the engineer follows DESIGN.md Section 37's decision hierarchy (reuse a component → reuse a pattern → reuse a token → reuse a principle → only then propose a new Design Decision) and documents the new decision back into DESIGN.md before merging — never silently in their own branch.
5. **Design Decisions already made for us that the frontend must respect exactly** (do not re-derive or “improve” these during implementation): Match Score is always shown with its five-signal breakdown inline, never behind a click (DESIGN.md Section 16); status is never color-only (DESIGN.md Section 18); the two-sided handover confirmation renders as two independent indicator dots, never one progress bar (DESIGN.md Section 26); Warm Sand is used only in Impact/Contribution contexts (DESIGN.md Section 4); Admin uses a visually distinct shell but the same token system (DESIGN.md Section 28).

Every task below lists its **Design dependencies** as specific DESIGN.md section numbers — an engineer opens that section before writing the component, not after.

06 — Frontend Domain Map

Engineer	Domain	Backend Dependency (Backend Plan §14)	Main Frontend Ownership
Engineer 1	Identity & Organizations + Shared Foundation	<code>/api/auth/*</code> , <code>/api/users/*</code> , <code>/api/organizations/*</code> , <code>/api/admin/users*</code>	App shell, routing skeleton, auth UI, session, guards, interceptors, layouts (public/app/admin), toasts, global error/loading infra, profile, organization UI
Engineer 2	Supply	<code>/api/categories*</code> , <code>/api/resources*</code>	Category selector, resource discovery/search/filter, resource detail, create/edit resource, resource lifecycle UI, "My Resources"
Engineer 3	Demand & Matching	<code>/api/requests*</code> , <code>/api/matches*</code>	Request discovery/search/filter, request detail, create/edit request, "My Requests", match generation UI, MatchScore visualization, accept/reject match
Engineer 4	Transfer, Trust & Impact	<code>/api/transactions/:matchId/confirm</code> , <code>/api/reports*</code> , <code>/api/users/me/contributions</code> , notifications (write-side is backend-only; MVP has no read/mark-as-read endpoint per Backend Plan Task 4.C, so Engineer 4's Notifications UI reads from whatever surface the backend does expose — see the Cross-Engineer Contract Gap noted in Section 07)	Handover detail + two-sided confirmation UI, contribution history, impact dashboard, notifications list, reports UI, verification-status display, trust indicators

Ownership boundary rule (mirrors Backend Plan Section 04): each engineer owns their feature folder under `features/` , their slice of `core/models/` , and any domain component in `shared/components/` that exists primarily to serve their domain (e.g., `MatchScoreComponent` is Engineer 3's, `ImpactCardComponent` is Engineer 4's) — but every primitive in `shared/ui/` and every file under `core/` (except domain models) is Engineer 1's and is only changed by others via a reviewed PR, exactly as the Backend Plan restricts changes to `authenticate.js` / `authorize.js` / `errorHandler.js` to Engineer 1.

Note on Categories: per Product Brief Section 15/24 and Backend Plan Section 05, Categories are Engineer 2's collection (read-only for the rest of the app, admin-write-only). The frontend `CategorySelectorComponent` / `CategoryComboboxComponent` therefore lives in Engineer 2's domain but is consumed as a shared, read-only input by Engineer 3's Request forms — this is documented explicitly in Section 07 as a cross-engineer contract, the same way the backend documents `categories` as Supply-owned but Demand-consumed.

07 — Cross-Engineer Contracts

No engineer silently changes a shared contract. Process, mirroring Backend Plan Section 12 exactly: **document the change** → **notify affected engineers** → **update the contract table below** → **update the implementation** → **update the tests**.

Contract	Owner	Shape source	Consumers	Versioning / breaking-change rule
AuthSession ({ user, token, isAuthenticated })	Engineer 1	Derived from Backend Plan §14 /api/auth/login response	All features (via AuthService)	Any field addition/removal requires a PR reviewed by all four engineers; consumed through AuthService getters only, never read from localStorage directly by feature code
User model	Engineer 1	Backend Plan §14, Product Brief §19	All	Owner-only edits
Organization model	Engineer 1	Backend Plan §14, Product Brief §19	Eng 1 (org UI), Eng 2/3 (VerificationBadge display on cards)	Owner-only edits
Category model	Engineer 2	Product Brief §19	Eng 2 (CRUD), Eng 3 (read-only, request category selector)	Owner-only edits; Eng 3 never writes a category field the model doesn't declare
Resource model	Engineer 2	Backend Plan §14, Product Brief §19	Eng 2, Eng 3 (MatchCard 's resource summary), Eng 4 (handover context)	Owner-only edits
Request model	Engineer 3	same	Eng 3, Eng 2 (n/a — one-directional), Eng 4	Owner-only edits
Match / MatchScoreBreakdown	Engineer 3	Backend Plan §14/§16 (scoreBreakdown)	Eng 3, Eng 4 (handover is created from an accepted match)	Owner-only edits; breakdown's five-signal fixed order (Category → Location → Quantity → Urgency → Availability, DESIGN.md §16) is part of the contract, not just a display choice
Handover model	Engineer 4	Backend Plan §14	Eng 4	Owner-only edits
Contribution model	Engineer 4	Backend Plan §14/§19	Eng 4	Owner-only edits
Notification model	Engineer 4	Product Brief §19	Eng 4	Owner-only edits — Cross-Engineer Contract Gap : the Backend Plan (Task 4.C) explicitly scopes notification read/mark-as-read endpoints as out of MVP HTTP surface (write-path only). Engineer 4 must confirm with the backend team, before starting Task 4.I, exactly what (if any) read endpoint exists at implementation time, and record the resolution here before building the Notifications page.
Report model	Engineer 4	Backend Plan §14/§19	Eng 4	Owner-only edits
ApiError ({ code, message, fieldErrors? })	Engineer 1	Backend Plan §16	All (via errorInterceptor)	Owner-only edits; every feature error-handling task in this document assumes this exact shape
Pagination ({ page, limit, total, totalPages })	Engineer 1	Backend Plan §14 (? page=&limit=)	Every list feature	Owner-only edits
Status enums (ResourceStatus , RequestStatus , MatchStatus , HandoverStatus , OrganizationVerificationStatus , ReportStatus)	Owning engineer per entity	Product Brief §11, Backend Plan §06	LifecycleTimeline , all status badges	Enum values copied verbatim from the Product Brief/DB Plan — no engineer adds a UI-only status value

Any additional gap discovered during implementation is appended as a new row here, tagged **Cross-Engineer Contract Gap**, before the discovering engineer's task is marked done.

08 — Engineer 1 — Shared Foundation

Mission: build everything every other engineer's feature code depends on before it can safely run — the app shell, routing skeleton, design-token wiring, auth, guards, interceptors, layouts, and global feedback infrastructure. This is Phase 1 in Section 30 and sits on the project's critical path (Section 32): nothing else can be demoed until this merges.

Prerequisite reading: DESIGN.md Sections 1–12 (principles through primitives), Backend Plan Section 07 (Authentication & Authorization) and Section 14 (/api/auth/* , /api/users/* , /api/organizations/* rows).

TASK 1.A — Angular Application Foundation

Owner: Engineer 1 · **Domain:** Shared Foundation

Mission: Stand up a clean Angular 17+ standalone-component application with strict TypeScript, Tailwind wired to the design tokens, and the base folder structure from Section 04.

Why it exists: every other task in this document assumes this structure already exists — it is the empty canvas the rest of the team paints on.

Prerequisites: none (this is the first task in the project).

Backend dependencies: none.

Design dependencies: DESIGN.md Section 32 (Tailwind Token Architecture) — the token file must exist before any component uses a semantic class.

Files/modules: angular.json , tsconfig.json ("strict": true), tailwind.config.js , src/design-tokens/tailwind-tokens.ts , src/styles.css , the full folder tree from Section 04 (empty placeholder files with a one-line comment are acceptable at this stage).

Implementation steps: 1. ng new dawwarha-frontend --standalone --style=css --routing and enable strict in tsconfig.json . 2. Install Tailwind (tailwindcss , postcss , autoprefixer) and initialize it against src/**/*.{html,ts} . 3. Create design-tokens/tailwind-tokens.ts transcribing every token in DESIGN.md Section 4 (colors), 6 (spacing — confirm Tailwind's default 4px-step scale already aligns, per Section 32's note), 9 (radius), 10 (shadows), 11 (motion) as named theme values. 4. Wire tailwind.config.js 's theme.extend to import from that file. 5. Create the folder tree from Section 04 with .gitkeep placeholders where no file exists yet. 6. Commit as the project's first commit on main ; create develop from it (Section 29).

Beginner explanation: *Why standalone components instead of NgModules?* Classic Angular required every component to be declared inside an NgModule — an extra file to keep in sync with every component you create. Standalone components remove that requirement: a component declares its own imports directly. For a four-person beginner team, one fewer file to remember to update per component is a real reduction in mistakes.

Expected behavior: ng serve boots a blank page styled with Tailwind's reset applied; a test div using bg-primary renders the correct green, proving the token pipeline works end to end.

Loading/Success/Error states: n/a (infrastructure task).

Validation: n/a.

Authorization: n/a.

Accessibility: confirm index.html 's <html lang="en"> / dir="ltr" are present as placeholders (Engineer 1 revisits this in Task 1.C once routing exists — full RTL wiring happens in Task 1.C alongside routing, since the dir attribute must respond to the active locale).

Responsive behavior: n/a at this stage — no screens exist yet.

Testing: ng test runs the CLI's default smoke test successfully; a manual check that changing a token value in tailwind-tokens.ts changes the rendered color confirms the pipeline is live, not hardcoded.

Acceptance criteria: repository builds clean from git clone , npm install , ng serve with zero console errors; Tailwind token pipeline verified working.

Common beginner mistakes: hardcoding a hex value in an early throwaway test component and forgetting to remove it before merging (defeats the entire point of Task 1.A); enabling strict mode later instead of now (retrofitting strict TypeScript onto a codebase with a dozen files already written is a much bigger task than starting with it).

Definition of Done: builds clean, token pipeline verified, folder structure matches Section 04, reviewed by one teammate, merged to develop .

Dependencies for other engineers: every subsequent task in this document.

TASK 1.B — Design System Foundation

Owner: Engineer 1 · **Domain:** Shared Foundation

Mission: Build the first wave of shared/ui/ primitives — Button, Input, Card, Badge, Skeleton, Toast, Spinner — exactly to DESIGN.md Section 12's contracts, so Engineers 2–4 have real components to compose from day one of Phase 2.

Why it exists: without this, each engineer would build their own button/card and the product would visually fragment within a week (DESIGN.md Section 2's "Consistency over novelty" principle exists precisely to prevent this).

Prerequisites: Task 1.A.

Backend dependencies: none.

Design dependencies: DESIGN.md Section 12 (full primitive-by-primitive spec), Section 4/9/10 (tokens these primitives consume).

Files/modules: shared/ui/button/ , shared/ui/input/ , shared/ui/card/ , shared/ui/badge/ , shared/ui/skeleton/ ,

`shared/ui/toast/` , `shared/ui/spinner/` — each a standalone component + its own spec file.

Implementation steps: 1. For each primitive, re-read its full contract in `DESIGN.md` Section 12 (Purpose, Anatomy, Variants, Sizes, States, Accessibility, Do/Don't) before writing code. 2. Implement typed `Input()`s for variant/size/state per Section 33's naming rule (`variant` , `size` for primitives — generic, not domain-specific names). 3. Implement `Output()`s as clear events (`clicked`) , not vague names). 4. Bake in accessibility per the primitive's contract (e.g., `Button` never suppresses its focus ring; `Skeleton` sets `aria-hidden` on its shapes and `aria-busy` on the containing region — `DESIGN.md` Section 12's `Skeleton` entry). 5. Write one Jasmine spec per primitive covering its documented states (default/hover/focus/disabled/loading where applicable).

Beginner explanation: *Why build a `Button` component instead of just using `<button class="...">` everywhere?* One component means one place to fix a bug, one place the focus ring is guaranteed correct, and one place that enforces the 40×40px touch-target rule (`DESIGN.md` Section 12) instead of forty places that might each forget it.

Expected behavior: every primitive renders all its documented variants/sizes/states correctly against the token system; a Storybook-style manual test page (a temporary route, removed before the final demo, or kept as an internal `/dev/components` route gated out of production builds) lets the team visually verify all seven primitives at once.

Loading states: `Skeleton` / `Spinner` themselves ARE the loading-state primitives — this task is where they're built, consumed by every later task's own "Loading states" section.

Success/Error states: `Toast` supports success/error/info variants per `DESIGN.md` Section 12's `Toast` entry.

Validation: n/a at the primitive level (validation is a Form-level concern, Section 16).

Authorization: n/a.

Accessibility: every primitive meets its individual contract's accessibility bullet from `DESIGN.md` Section 12 — this is verified per-primitive in code review, not assumed.

Responsive behavior: primitives are inherently responsive (no fixed pixel widths beyond icon/touch-target sizing); verified at mobile/desktop widths in the dev test page.

Testing: one spec file per primitive; each documented state gets at least one test asserting the correct CSS class/attribute is applied.

Acceptance criteria: all seven primitives pass their specs; a design-review pass by the team confirms each matches its `DESIGN.md` Section 12 entry with no deviation.

Common beginner mistakes: adding a prop that isn't in the documented Variants/Sizes list "because it seemed useful" (Section 33/40's rule against open, undocumented APIs); copy-pasting Bootstrap-style button CSS instead of building from tokens (an explicit Design Anti-Pattern, `DESIGN.md` Section 36).

Definition of Done: all seven primitives built, tested, reviewed, merged; a short "how to use `shared/ui/`" note added to the frontend README (Section 38).

Dependencies for other engineers: every domain component built from Task 1.C onward across all four engineers.

(Remaining primitives — `Textarea`, `Select`, `Combobox`, `Checkbox`, `Radio`, `Switch`, `Search`, `File Upload`, `Avatar`, `Tooltip`, `Dropdown`, `Dialog`, `Drawer`, `Tabs`, `Alert`, `Table`, `Pagination` — are built incrementally as the first feature that needs each one requires it, still under Engineer 1's ownership per Section 06, tracked as sub-items of this task in the checklist, Section 40.)

TASK 1.C — Routing Architecture

Owner: Engineer 1 · **Domain:** Shared Foundation

Mission: Define the full lazy-loaded route tree (Section 24) with placeholder route components for every feature, wired to the three layout shells (`public/app/admin`), and set `<html dir= / lang` reactively for RTL/LTR (`DESIGN.md` Section 29).

Prerequisites: Task 1.A.

Backend dependencies: none.

Design dependencies: `DESIGN.md` Section 23 (Navigation), Section 29 (RTL/LTR sidebar-mirroring rule).

Files/modules: `app.routes.ts` , one route file per feature (`features/*/*.routes.ts`), `layout/public-layout/` , `layout/app-layout/` , `layout/admin-layout/` .

Implementation steps: 1. Transcribe the full route map from Section 24 into `app.routes.ts` , using `loadComponent` / `loadChildren` for every feature branch. 2. Nest routes under the correct layout: public routes render inside `PublicLayoutComponent` , authenticated routes inside `AppLayoutComponent` , admin routes inside `AdminLayoutComponent` — three separate route sub-trees, per `DESIGN.md` Section 23's "never a single merged menu" rule. 3. Build a `LanguageService` (or extend `AuthService`) that holds the active locale, applies `document.documentElement.setAttribute('dir', ...)` and `lang` , and persists the choice. 4. Add a placeholder component per feature route (`<h2>Resources — coming soon</h2>`) so the route tree is fully clickable from day one, letting Engineers 2–4 replace placeholders incrementally without touching routing config.

Beginner explanation: *What is lazy loading and why does it matter here?* Without it, the browser would download the code for Admin, Matches, Handovers, and every other feature before the user even logs in — slow, and wasteful for a visitor who only browses the public pages. `loadComponent` / `loadChildren` tell Angular "only download this feature's code the first time someone actually navigates there."

Expected behavior: every route in Section 24 is reachable and renders inside the correct shell; switching the language toggle flips `dir` and visibly mirrors the sidebar (once Task 1.M exists).

Loading states: a thin top progress bar (per `DESIGN.md` Section 10's "page transitions: none by default... thin top progress bar only if >300ms") appears during lazy-chunk loading on slow connections — implemented once here via a router-event listener, not per-route.

Error states: an unmatched route (`**`) renders a simple "Page not found" component, distinct from the 404 detail-page state (Section 17 covers the latter).

Validation: n/a.

Authorization: guards are wired in Task 1.G, not here — this task's routes are unguarded placeholders; guard wiring is a separate, reviewable diff.

Accessibility: route changes move focus to the new page's main heading (a small RouterOutlet -adjacent focus-management utility) so keyboard/screen-reader users aren't stranded at the old scroll position — required per DESIGN.md Section 30's keyboard-navigation bullet, easy to forget in an SPA.

Responsive behavior: the three layout shells each define their own responsive nav collapse (detailed in Tasks 1.L/1.M/1.N).

Testing: a router test confirming every route in Section 24 resolves to a component without a 404; a test confirming dir flips correctly when the locale changes.

Acceptance criteria: full route tree navigable; RTL toggle works app-wide; no route requires a guard to be present yet (guards land in 1.G).

Common beginner mistakes: eagerly importing every feature's route file at the top of app.routes.ts instead of using LoadComponent (silently defeats lazy loading — bundle size looks fine in dev, only shows up as a real problem in a production build analysis); forgetting to set dir on <html> and only setting it on a wrapper <div> (breaks native form/input directionality that depends on the root dir).

Definition of Done: full route tree present and tested; merged before any Phase 2 feature work begins.

Dependencies for other engineers: every feature route from Section 24 onward.

(Tasks 1.D through 1.P — Authentication UI, Auth Session Management, HTTP Interceptor, Route Guards, Role-Based Navigation, Global Error Handling, Global Loading System, Toast Infrastructure, Public Layout, Authenticated App Layout, Admin Layout, User Profile, Organization UI — are executed with the same full template above. Each is summarized below with its core contract; the full write-up is produced by Engineer 1 at execution time following the Task 1.A–1.C pattern exactly.)

Task	Mission	Design dependency	Backend dependency	Key acceptance criterion
1.D — Authentication UI	Login + Register pages, Reactive Forms per DESIGN.md §19	§19 (Forms), §35 (Authentication template)	POST /api/auth/register , POST /api/auth/login	Field-level errors render inline; server 400s map to the correct field via ApiError.fieldErrors
1.E — Auth Session Management	AuthService holds AuthSession , persists the JWT (e.g. in-memory + a single storage key), restores session on refresh, exposes isAuthenticated / currentUser as signals	§23 (role-aware nav depends on this)	GET /api/users/me (session restore)	Refreshing the page while logged in does not log the user out
1.F — HTTP Interceptor	authInterceptor attaches Authorization: Bearer <token> to every outgoing request except /auth/register / /auth/login	—	Backend Plan §07.1 (Authorization: Bearer contract)	No protected request is ever sent without a token when one exists
1.G — Route Guards	authGuard (must be logged in), roleGuard(. . . roles) , orgVerifiedGuard	§23	Backend Plan §07.2 permission matrix	An unauthenticated user hitting a protected route is redirected to Login with a return-url; matches Backend Plan's exact 401 copy ("Please log in to continue")
1.H — Role-Based Navigation	Sidebar/bottom-tab nav items shown/hidden per role, matching §23 exactly (Public/User/Organization/Admin nav sets)	§23	— (derived from AuthSession.user.role)	A user role never sees Admin nav items, even briefly during a race condition on load
1.I — Global Error Handling	errorInterceptor normalizes every HTTP error into ApiError ; unhandled 500s trigger a generic toast, never a raw stack trace	§12/§18/§19 Alert & Toast primitives	Backend Plan §16 (exact error shape/codes)	A simulated 500 never shows raw backend text to the user
1.J — Global Loading System	Route-level top progress bar (from 1.C) + a documented convention every feature follows for list/detail loading (Skeleton) vs. button loading (inline spinner)	§10/§12/§20	—	No feature ships a full-page spinner overlay (an explicit Design Anti-Pattern)
1.K — Toast/Notification Infrastructure	ToastService singleton queue, max one visible at a time (others queue), 4s auto-dismiss, manually dismissible	§12 Toast entry	—	Two toasts fired in quick succession never overlap on screen
1.L — Public Layout	Top nav (Home/Browse Resources/Browse Requests/About/Login/Register), hamburger on mobile with Login/Register pinned visible	§23 "Public"	—	Matches §23's exact nav item list and mobile-pinned-buttons rule
1.M — Authenticated App Layout	Sidebar (desktop/laptop) / 5-item bottom tab bar + Notifications icon (mobile), Organization section shown distinctly when applicable	§23 "User"/"Organization"	—	7-item nav correctly collapses to the documented 5-item mobile set, never all 7 crammed in
1.N — Admin Layout	Visually distinct shell (denser header, separate sidebar) per §28	§28	—	An admin switching to their personal-user context sees a visually unmistakable shell change, never a merged menu

1.O — User Profile	Editable profile form, GET/PUT <code>/api/users/me</code>	\$19, \$35 "Profile"	GET/PUT <code>/api/users/me</code>	Save shows an explicit success Toast — never a silent success (§34 Do/Don't)
1.P — Organization UI	Org registration/profile/verification- status display	\$13 OrganizationCard, \$25 Trust states	POST <code>/api/organizations</code> , GET/PUT <code>/api/organizations/:id</code>	Pending/rejected verification states shown only to the org's own dashboard, never to public browsers of the org's profile — matching Backend Plan §17's privacy rule exactly

09 — Engineer 2 — Supply

Mission: everything a provider needs to publish and manage surplus — Categories (read-side; write-side is admin, itself under Engineer 2's ownership since Categories is a Supply collection), Resource discovery, and the full Resource lifecycle UI.

Prerequisite reading: DESIGN.md Sections 14 (Resource Card), 21 (Empty States), 22 (Error States); Backend Plan Section 09 row for resources, Section 14's Supply rows.

TASK 2.A — Category Data Layer & Selector

Owner: Engineer 2 · **Domain:** Supply

Mission: Build `CategoryApiService` and a reusable `CategorySelectorComponent` / `CategoryComboboxComponent` consumed by both Resource and Request creation forms (the latter by Engineer 3, per Section 07's cross-engineer contract).

Why it exists: the category taxonomy is controlled (Product Brief §15/§24 — admin-managed, never free text); every form that lets a user pick a category must pick from the same live list, not a hardcoded array duplicated in two places.

Prerequisites: Task 1.B (Combobox primitive, once built), Task 1.F (auth interceptor, for admin-only writes).

Backend dependencies: `GET /api/categories` (public), `POST/PUT/DELETE /api/categories/:id` (admin-only, Backend Plan §14).

Design dependencies: DESIGN.md §12 Combobox entry (search + select from a longer list — categories can grow).

Files/modules: `features/categories/category-api.service.ts`, `core/models/category.model.ts`, `shared/components/category-selector/` (consumed cross-domain).

Implementation steps: 1. Define `Category { id, name, description, isActive }` in `core/models/category.model.ts` per Product Brief §19. 2. `CategoryApiService.list()` → `GET /api/categories`, cached in a `BehaviorSubject` / signal for the session (categories change rarely; re-fetch only after an admin write, not on every form open). 3. Build `CategorySelectorComponent` on top of the Combobox primitive: debounced local filtering (categories list is small enough that server-side search isn't needed for MVP — filtering happens client-side against the already-fetched list), single-select. 4. Admin-only `CategoryAdminListComponent` for CRUD (Task 2.B, listed below) reuses the same `CategoryApiService`.

Beginner explanation: *Why not just let each form hardcode its own list of categories?* If the admin adds "Electronics" as a new category, every hardcoded copy would need a manual code change and redeploy. Fetching from `GET /api/categories` means the taxonomy is data, not code — the whole point of Product Brief §24's "controlled category taxonomy administrable only by Admin."

Expected behavior: the selector shows the live category list, is fully keyboard-operable, and both Resource and Request creation forms use the exact same component instance type (not two near-identical copies).

Loading states: Skeleton row placeholders while the list fetches on first use.

Success states: n/a (selection has no separate confirmation step — it's part of a larger form).

Error states: if the category fetch fails, the selector shows an inline "Couldn't load categories — retry" affordance rather than silently rendering empty (this would otherwise look like "no categories exist," which is factually wrong and would block every resource/request from being created).

Validation: required-field validation lives in the consuming form (Resource/Request creation), not in the selector itself — the selector only guarantees it never offers an inactive (`isActive: false`) category as selectable.

Authorization: read is public; admin CRUD is guarded by `roleGuard(['admin'])` at the route level (Task 2.B).

Accessibility: WAI-ARIA combobox pattern per DESIGN.md §12 — `aria-expanded`, `aria-activedescendant`, announces result-count changes on filter.

Responsive behavior: becomes a bottom sheet on mobile per the Combobox primitive's documented responsive behavior.

Testing: unit test for the filtering logic; integration test confirming a fetched category list renders selectable options; a11y test for keyboard navigation (arrow keys select, Escape closes).

Acceptance criteria: one selector component, two consumers (Resource form, Request form), zero duplicated category-fetching logic.

Common beginner mistakes: each domain writing its own `getCategories()` call instead of sharing `CategoryApiService` (breaks the single-source-of-truth cache and risks the two forms showing different lists momentarily).

Definition of Done: built, tested, consumed identically by both Resource and Request forms once Engineer 3 reaches Task 3.F; documented in Section 07 as a cross-engineer shared component.

Dependencies for other engineers: Engineer 3's Task 3.F (Create Request) directly imports this component.

TASK 2.D — Resource Discovery Page

Owner: Engineer 2 · **Domain:** Supply

Mission: Build the public `Browse Resources` page: a filterable, paginated grid of `ResourceCard`s per DESIGN.md Section 14 and Section 37's "Browse" template.

Why it exists: this is the seeker-facing half of the core product loop (FR-005/FR-022) and the first real screen most anonymous visitors will judge the product by.

Prerequisites: Task 2.A (categories for the filter bar), Task 1.B (Card/Badge/Skeleton primitives).

Backend dependencies: `GET /api/resources` with `category`, `city/area`, `status`, `page`, `limit` query params (Backend Plan §14, §04's exact query-filter shape).

Design dependencies: DESIGN.md §14 (Resource Card anatomy/states), §31 (Search/Filter System), §35 "Browse" template, §21 "No search

results" empty state.

Files/modules: `features/resources/resource-discovery/` , `features/resources/resource-api.service.ts` , `shared/components/resource-card/` .

Implementation steps: 1. Build `ResourceApiService.list(filters, page, limit)` typed against `Resource` (Section 07) and `Pagination` . 2. Build `ResourceCardComponent` exactly per `DESIGN.md` §14's six-level anatomy (Category → Quantity → Location → Availability → Description → metadata), horizontal on desktop / stacked on mobile, hover lift on desktop only. 3. Build the filter bar: `LocationChip` -style area filter, `CategorySelectorComponent` (from Task 2.A) as a filter (interactive variant), availability toggle — capped at the Design System's six-filter-dimension limit (`DESIGN.md` §31). 4. Wire "Load More" pagination (not numbered pagination — §12 `Pagination` primitive's explicit rule for public Browse screens). 5. Wire the debounced (300ms) search input. 6. Implement the loading (skeleton cards matching real anatomy), empty ("No resources match these filters" + Clear Filters Ghost button, §21), and populated states.

Beginner explanation: *Why debounce the search input?* Without debouncing, every keystroke fires an HTTP request — typing "furniture" would fire 9 requests, 8 of them wasted. Debouncing waits 300ms after the user stops typing before firing one request.

Expected behavior: filtering/searching updates the grid without a full page reload; clearing filters resets to the unfiltered list in one tap; the whole card is a single click target to the detail page (Task 2.G).

Loading states: 3–6 skeleton `ResourceCard` s matching exact anatomy on first load and on filter change (per `DESIGN.md` §26/§20).

Success states: n/a (browsing has no "success" moment — arriving at results is the state).

Error states: a failed fetch shows a page-level Alert banner ("Something went wrong. Please try again.") — never a blank grid with no explanation.

Validation: query params are constructed from typed filter state, never string-concatenated from raw user input (mirrors the backend's own NoSQL-injection-prevention discipline, Backend Plan §15 — the frontend's job here is correctness/UX, not security, since the backend remains the trusted enforcement point, but malformed params should still never be sent).

Authorization: none — public page.

Accessibility: filter chips are real `<button>` s with `aria-pressed` ; result count changes are announced via `aria-live="polite"` near the search input.

Responsive behavior: 1 column mobile → 2 tablet → 3 laptop → 3–4 desktop, per `DESIGN.md` §8's grid table; filters move into a bottom-sheet drawer below tablet width.

Testing: component test for `ResourceCardComponent` 's anatomy; integration test for filter-triggers-refetch; empty-state test; pagination "Load More" test.

Acceptance criteria: matches `DESIGN.md` §14/§31/§37 exactly; Lighthouse a11y score check on this page specifically (it's the most-trafficked public page).

Common beginner mistakes: truncating category/quantity/status in the card layout to "make it fit" — `DESIGN.md` §5 explicitly forbids ever truncating these fields; building a second bespoke card instead of reusing `ResourceCardComponent` on a different page later (Task 2.K's "My Resources" must reuse this same component, not duplicate it).

Definition of Done: full Section 39 Definition of Done checklist passes for this page specifically.

Dependencies for other engineers: `ResourceCardComponent` is reused by Engineer 3's `MatchCard` (compact resource summary) and by Engineer 2's own Task 2.K.

(Tasks 2.B–2.C, 2.E–2.P — *Category Admin CRUD, Search/Filter helper, Resource Search, Resource Filtering, Resource Details, Create Resource, Edit Resource, Resource Lifecycle UI, My Resources, Empty/Loading/Error States, Responsive UX, Accessibility* — follow the same template. Condensed summary:)

Task	Mission	Key backend dependency	Key acceptance criterion
2.B — Category Admin CRUD	Admin-only list/create/edit/deactivate categories	<code>POST/PUT/DELETE /api/categories/:id</code>	Delete is soft (<code>isActive:false</code>), never a hard remove — matches Backend Plan §12.5
2.C — Resource API Service (full)	Complete typed service: list/get/create/update/status-transition/delete	<code>GET/POST/PUT/DELETE /api/resources*</code> , <code>PUT /api/resources/:id/status</code>	Status changes call a distinct <code>transitionStatus(id, action)</code> method — never a raw <code>PUT</code> with a <code>status</code> field in the body, mirroring Backend Plan §09/§12.2's "the ONE place status changes happen" rule at the API-consumption layer too
2.E — Resource Search	Debounced search box wired into 2.D	<code>GET /api/resources?search=</code> (if backend supports free-text; else client-side title match against fetched category/location-filtered results — confirm with backend, log as a Contract Gap if unresolved)	Search never fires more than one request per 300ms pause
2.F — Resource Filtering	Category/location/availability filter logic	same as 2.D	Never more than the 6 documented filter dimensions exposed
2.G — Resource Details	Full detail page per <code>DESIGN.md</code> §37 "Detail" template	<code>GET /api/resources/:id</code>	Unavailable/Expired states show the explanatory note per <code>DESIGN.md</code> §14
2.H — Create Resource	Guided multi-section Reactive Form	<code>POST /api/resources</code>	Matches <code>DESIGN.md</code> §19's "guided form" direction — sectioned, not one flat field list
2.I — Edit Resource	Same form, pre-filled, blocked on terminal states	<code>PUT /api/resources/:id</code>	A 409 from editing a terminal-state resource shows the exact Backend Plan §16 message
2.J — Resource Lifecycle UI	<code>LifecycleTimeline</code> wired to	— (display only)	Uses the exact 8-step sequence from

	Resource.status		Product Brief §11 — no invented states
2.K — My Resources	Provider's own listings, reusing ResourceCardComponent	GET /api/resources?providerId=me (or however Backend Plan's ownership-filtered list is exposed — confirm exact query shape, log as Contract Gap if undocumented)	Reuses Task 2.D's card, never a duplicate
2.L — Resource Empty States	All Resource-domain empty states	—	Matches DESIGN.md §21 table exactly
2.M — Resource Loading States	Skeleton coverage audit across all Resource screens	—	No screen falls back to a generic spinner where a skeleton is specified
2.N — Resource Error Handling	400/403/404/409 mapping specific to Resource endpoints	Backend Plan §16	Every Resource-domain error code has a mapped, tested UI treatment
2.O — Resource Responsive UX	Verified mobile/tablet/desktop pass across all Resource screens	—	Manually verified at all breakpoints in DESIGN.md §8, not assumed
2.P — Resource Accessibility	Full a11y audit of the Resource domain	DESIGN.md §30	Keyboard-only walkthrough of publish → edit → cancel succeeds with no mouse

10 — Engineer 3 — Demand & Matching

Mission: the demand side (Requests) and the platform's flagship trust surface — the explainable, rule-based Matching UI. DESIGN.md §16 and Product Brief §12/§23 both single this out as the feature that must never feel like “mysterious AI.”

Prerequisite reading: DESIGN.md Sections 15 (Request Card), 16 (Matching UI/MatchScore), Backend Plan Section 10 (Task 3.A/3.B — the transactional accept endpoint), Product Brief Section 12 (scoring model).

TASK 3.A — Request API Service & CRUD

Owner: Engineer 3 · **Domain:** Demand & Matching

Mission: Mirror Engineer 2's Resource CRUD pattern for Requests — same rigor, demand-side fields.

Prerequisites: Task 2.A (`CategorySelectorComponent` , shared cross-engineer per Section 07), Task 1.F.

Backend dependencies: `GET/POST/PUT/DELETE /api/requests*` , `PUT /api/requests/:id/status` (Backend Plan §14).

Design dependencies: DESIGN.md §15 (Request Card — same anatomy pattern as Resource Card, re-prioritized with Urgency).

Files/modules: `features/requests/request-api.service.ts` , `core/models/request.model.ts` , `shared/components/request-card/` .

Implementation steps: 1. Define `Request { id, category, quantity, urgency, location, status, requesterId, ... }` per Product Brief §19; urgency: 'low' | 'medium' | 'high' exactly. 2. `RequestCardComponent` built on the same `Card` primitive as `ResourceCard` , re-ordered per DESIGN.md §15: Category → Requested quantity → Location → `UrgencyBadge` → Status. 3. CRUD methods mirroring `ResourceApiService` 's shape (`list/get/create/update/transitionStatus/delete`) so the two services are structurally interchangeable for anyone reading the code. 4. Reuse `CategorySelectorComponent` from Task 2.A directly — do not fork it.

Beginner explanation: *Why build this so similarly to Engineer 2's Resource code?* DESIGN.md §2's “Consistency over novelty” principle and §15's explicit note that Request Card “mirrors the Resource Card exactly” — the anatomy pattern is shared, only the specific fields (urgency instead of availability-only) differ. Writing it structurally the same way makes the whole codebase easier for any of the four engineers to read, not just its author.

Expected behavior: full Request CRUD works end to end with the exact same UX quality bar as Resources.

Loading/Error/Empty states: identical pattern to Resources (Task 2.D/2.L/2.N), applied to Requests.

Validation: urgency is a closed enum (segmented control or select, never free text) per DESIGN.md §31.

Authorization: owner/admin for write; per Backend Plan §14, `GET /api/requests` requires auth (unlike Resources, which is public read) — confirmed from Backend Plan's Section 14 table row exactly (`GET /api/requests - auth -`), not assumed to be public by analogy with Resources.

Accessibility/Responsive: same bar as Resource Card (Task 2.P/2.O equivalent).

Testing: mirrors Task 2.A's/2.C's test structure applied to the Request transition table (Product Brief §11 — the state machine is the same named lifecycle, applied to both entities).

Common beginner mistakes: copy-pasting `resourceLifecycleService` -equivalent frontend logic instead of writing request-specific status-transition handling against the request-specific action set — the PATTERN is shared, the DATA (which actions are valid from which status) is not, mirroring Backend Plan Task 3.A's own explicit warning about this exact mistake on the backend side.

Definition of Done: Request CRUD reaches parity with Resource CRUD; `RequestCardComponent` ready for Task 3.B/3.C reuse.

Dependencies for other engineers: `RequestCardComponent` 's compact variant is reused inside `MatchCardComponent` (Task 3.L).

TASK 3.K — Match Results & Score Visualization (flagship task)

Owner: Engineer 3 · **Domain:** Demand & Matching

Mission: Build `MatchCardComponent` and `MatchScoreComponent` exactly per DESIGN.md Section 16 — the platform's single most important trust surface.

Why it exists: Product Brief §12/§23 and DESIGN.md §16 are unambiguous: a match score shown without its explanation is unacceptable anywhere in the product. This is not a “nice to have” polish item — it is the literal difference between Dawwarha and “mysterious AI.”

Prerequisites: Task 3.A (Request Card), Task 2.D (Resource Card, for the compact summary), matching endpoint live on the backend (Backend Plan Task 3.B).

Backend dependencies: `POST /api/matches/:resourceId/generate` , `GET /api/matches` , `PUT /api/matches/:id/accept` , `PUT /api/matches/:id/reject` (Backend Plan §14; `scoreBreakdown` shape from Backend Plan §06/§14).

Design dependencies: DESIGN.md §16 in full — this is the section's entire reason for existing.

Files/modules: `shared/components/match-card/` , `shared/components/match-score/` , `features/matches/match-api.service.ts` , `core/models/match.model.ts` .

Implementation steps: 1. Define `MatchScoreBreakdown { category: boolean, location: boolean, quantity: boolean, urgency: boolean, availability: boolean }` matching Backend Plan's scoring signal order **exactly** — Category → Location → Quantity → Urgency → Availability, never reordered, per DESIGN.md §16's explicit fixed-order requirement. 2. Build `MatchScoreComponent` : large H3-weight percentage at the top, breakdown rendered immediately below it in the SAME component render — never behind a “why?” click-through, never a separate route/tab. 3. Score-range visual treatment: 85–100% → `success` token; 60–84% → `info` token — implemented as a pure display mapping over the numeric score, not a re-derivation of the underlying threshold logic (that stays backend-owned per DESIGN.md 's explicit Design Decision that this is UI-layer only). 4. Whole-number rounding only — never render more than integer precision, even if the API were to return a decimal (defensive `Math.round()` at the display layer). 5. Confidence copy is hardcoded to the exact factual sentence form: “92%

match based on category, location, quantity, urgency, and availability." — never "our AI thinks..." anywhere in the codebase. 6. `aria-label` on the score element reads the full sentence form per `DESIGN.md §16`'s accessibility requirement ("92 percent match: category matched, location matched, ..."). 7. Mobile: breakdown collapses to a 2-column grid of the five signals (CSS grid, not a taller single column) — every signal stays visible without a tap, per `DESIGN.md §16`'s explicit mobile rule. 8. Build `MatchCardComponent`: `MatchScoreComponent` header → compact `ResourceCardComponent` / `RequestCardComponent` summaries (category/quantity/location only — pass an `variant="compact"` input) → Accept (Primary)/Reject (Ghost, danger-toned) actions. 9. Desktop: two-column Resource/Request comparison beneath the score. Mobile: stacked, Resource above Request.

Beginner explanation: *Why is the breakdown never allowed to be hidden behind a click?* Imagine seeing "92% Match" with zero explanation — you'd have no way to judge whether the system actually understood your need or is guessing. Showing all five signals immediately, always, is what makes the score *trustworthy* instead of just a number. This single design rule is arguably the most important one in the entire design system for this product.

Expected behavior: generating matches against an available resource returns a ranked list, each rendered as a full `MatchCardComponent` with its score and breakdown immediately visible — no additional interaction required to see why a match scored the way it did.

Loading states: "Match generation" shows skeleton Match Cards — a skeleton score numeral + skeleton breakdown rows — per `DESIGN.md §26`'s explicit "communicates computing matches without inventing fake progress percentages" rule; never a fake animated progress bar.

Success states: Accept shows an explicit success Toast/transition, not a silent status change (Section 17/§34 Do/Don't).

Error states: generating against a non-available resource → 409, shown as an inline banner near the trigger action, using the Backend Plan's exact returned message where one is specified (Section 17).

Validation: n/a beyond the accept/reject action guards (button disabled while a request is in flight, to prevent a double-submit race).

Authorization: Accept/Reject restricted to a party to the match (enforced server-side per Backend Plan §07.2 — the frontend hides the buttons for a non-party as a UX courtesy per Section 13's "UX boundary, not the authoritative boundary" principle, never as the actual security control).

Accessibility: every breakdown row pairs its check/cross icon with a visible text label — never icon-only (`DESIGN.md §16/§28`); the score's `aria-label` is tested explicitly with a screen-reader-emulation test.

Responsive behavior: exactly as described in steps 7 and 9 above — this is one of the few components in the whole app with an explicitly documented, non-default mobile layout (2-column breakdown grid) rather than simple reflow.

Testing: unit test verifying the five-signal order never changes regardless of API response key order; unit test verifying whole-number rounding; unit test verifying the `aria-label` sentence construction; integration test for the full generate → display → accept flow; a specific regression test asserting the breakdown is present in the DOM immediately on render (not behind an interaction) — this test exists specifically to prevent a future regression of the single most important design rule in this component.

Acceptance criteria: matches `DESIGN.md §16` with zero deviation; passes the "explain it to someone unfamiliar with the product in one sentence" test that `DESIGN.md §33` (Trust & Verification UI, restated for Matching) implicitly demands of every trust-related UI element.

Common beginner mistakes: putting the breakdown in a tooltip or an expandable "details" accordion "to save space" — this directly violates `DESIGN.md §16/§20`'s core rule and must be caught in code review every time; reordering the five signals to match whatever order the API happens to return them in, instead of enforcing the fixed Category → Location → Quantity → Urgency → Availability order in the component itself.

Definition of Done: all tests above pass, including the explicit "breakdown never hidden" regression test; PR reviewed with special attention from the whole team given this component's product importance, not just the usual single reviewer.

Dependencies for other engineers: Engineer 4's Handover UI (Task 4.B) links back to the originating `Match` for context; the accepted-match transition is what triggers Engineer 4's domain to begin (Section 31's dependency graph).

(Remaining Engineer 3 tasks — 3.B Request Discovery, 3.C Request Search, 3.D Request Filters, 3.E Request Details, 3.F Create Request, 3.G Edit Request, 3.H Request Lifecycle UI, 3.I My Requests, 3.J Matching API Integration, 3.L Match Explanation — covered by 3.K above, 3.M — folded into 3.K, 3.N Accept Match, 3.O Reject Match, 3.P Match Loading/Error/Empty States — follow the same template as Engineer 2's condensed list in Section 09, applied to the Demand domain, with 3.N/3.O calling `MatchApiService.accept(id) / .reject(id)` and handling the specific 403/409 cases from Backend Plan §14's Match rows.)

11 — Engineer 4 — Transfer, Trust & Impact

Mission: the two-sided handover confirmation, contribution/impact display, notifications, reports, and trust/verification indicators. Also owns cross-module integration testing and end-to-end UX consistency, mirroring the Backend Plan's own assignment of this role to the equivalent backend engineer (Backend Plan Section 11).

Prerequisite reading: DESIGN.md Sections 17 (Lifecycle), 25 (Trust & Verification), 26 (Handover UX), 27 (Contribution & Impact); Backend Plan Section 11 (Task 4.A — the two-sided confirmation endpoint contract).

TASK 4.C — Handover Lifecycle UI & Two-Sided Confirmation (flagship task)

Owner: Engineer 4 · **Domain:** Transfer, Trust & Impact

Mission: Build the Handover detail page implementing FR-013's two-sided confirmation model as a structurally visible UI, exactly per DESIGN.md Section 26's Design Decision — two independent indicator dots, never a single progress bar.

Why it exists: this is where the platform's core trust promise ("a transfer is not complete until both sides confirm it," Product Brief §08) either reads clearly to the user or doesn't. DESIGN.md §26 explains precisely why a single progress bar would be actively misleading here (a bar filling halfway reads as "50% done," not "one side confirmed, structurally independent of the other").

Prerequisites: Task 3.K (an accepted Match is what creates a Handover — Backend Plan §10/§13's transaction), Task 1.K (Toast infrastructure).

Backend dependencies: POST /api/transactions/:matchId/confirm (Backend Plan §14/§11 — note the endpoint is keyed by matchId, not a separate handover ID, exactly as Backend Plan Task 4.A specifies).

Design dependencies: DESIGN.md §26 in full, §17 (Lifecycle Timeline component), §21 handover-adjacent empty states.

Files/modules: features/handovers/handover-detail/, features/handovers/handover-api.service.ts, core/models/handover.model.ts, shared/components/lifecycle-timeline/.

Implementation steps: 1. Define Handover { id, matchId, providerId, seekerId, confirmedByProvider, confirmedBySeeker, status, completedAt, ... } per Product Brief §19/Backend Plan §06. 2. Build LifecycleTimelineComponent per DESIGN.md §17: Completed (filled success circle), Active (filled primary, pulsing, respecting reduced-motion), Upcoming (hollow, border color), Cancelled (text-muted dash), Failed (danger x) — horizontal on desktop, vertical stepper on mobile with the active step scrolled into view. 3. Inside the "In Handover" timeline step specifically, render **two independent indicator dots/checks** — one for confirmedByProvider, one for confirmedBySeeker — never a merged single progress element (DESIGN.md §26's Design Decision, implemented literally). 4. Determine the viewing user's side (provider / seeker) client-side purely for display purposes (which dot is "yours") — the actual side used in the API call is derived server-side from the authenticated user, per Backend Plan Task 4.A Step 2's explicit rule that "side" is never client-supplied; the frontend does not send a side field in the confirm request body at all. 5. "Confirm Handover" is a large, unmistakable Primary action, disabled/hidden once the current viewer's own side is already confirmed (idempotent on the backend regardless, per Backend Plan Task 4.A, but the frontend still shows "Waiting on the other party" rather than inviting a redundant tap). 6. On the response indicating status: 'completed', render an explicit, unmistakable success state — not just the badge color changing — per Product Brief §23/DESIGN.md §34's "never a silent success" rule; this likely means a dedicated success panel/illustration-free confirmation block, plus a link to the now-updated Contribution History (Task 4.G). 7. Failed/Cancelled states show the plain-language explanation and next-step copy per DESIGN.md §17/Product Brief §11's edge cases (resource/request returns to Available).

Beginner explanation: Why does the frontend never send which "side" is confirming? If the browser sent { side: 'provider' } in the request body, anyone could edit that value and falsely confirm on behalf of the other party — completely defeating the two-sided guarantee. Instead, the backend looks at who is logged in (via the JWT) and figures out their side itself. The frontend's job is only to show the right button to the right person — the actual trust guarantee lives entirely on the backend, and the frontend must never try to "help" by sending side information the backend doesn't need and shouldn't trust.

Expected behavior: provider confirms → their dot fills, seeker's page (on next load/poll) shows "Waiting on your confirmation" language for the still-open side; seeker then confirms → both dots fill, status flips to Completed, and an unmistakable success state renders for both parties.

Loading states: the Confirm action shows the Button primitive's loading state (Section 12/DESIGN.md §12) while the request is in flight; the page itself shows a skeleton matching the real detail-page section layout on initial load.

Success states: the completion state described in step 6 — this is the single most important "success state must be explicit" example in the whole app, since it is the moment the entire product's core promise is fulfilled.

Error states: a non-party attempting to view/confirm → 403, handled per Section 17's mapping; confirming an already-cancelled/no-show handover → 409 with the specific returned message.

Validation: n/a beyond disabling the Confirm button during an in-flight request (prevents a double-submit — though the backend is documented as idempotent regardless, Backend Plan Task 4.A, so this is a UX nicety, not a correctness requirement).

Authorization: page itself is guarded (authGuard); the Confirm action is additionally hidden if the viewer is neither providerId nor seekerId on this handover (again, UX-only — the real boundary is server-side, Section 13).

Accessibility: the two independent indicators each carry a visible text label ("You confirmed" / "Waiting on provider"), never conveyed by dot color/fill alone (DESIGN.md §18/§28's "never color alone" rule applied here specifically); the completion success state uses role="status" so it's announced to screen readers without stealing focus destructively.

Responsive behavior: Lifecycle Timeline flips from horizontal to a vertical stepper below tablet width per DESIGN.md §8/§17; the two-dot indicator stays legible and correctly labeled at every width — this is explicitly called out because a cramped mobile layout is exactly where a design might be tempted to collapse the two indicators back into a single bar, which must not happen.

Testing: integration test simulating provider-confirms-then-seeker-confirms, asserting the UI never shows "Completed" until both flags are true; a test asserting the confirm request body never contains a side field; a test asserting a double-tap of Confirm from the already-confirmed side does not show an error state (matches the backend's documented idempotency, Backend Plan Task 4.A).

Acceptance criteria: matches DESIGN.md §26 exactly, including the specific two-independent-dots requirement; the success state is unmistakable per manual design review.

Common beginner mistakes: rendering the two confirmation flags as a single combined progress bar “because it looks cleaner” — this is the single most explicitly-warned-against mistake in this entire task, directly contradicting a named Design Decision; accidentally sending a `side` field in the request body because it “seemed like the API would want to know” (Backend Plan Task 4.A names this exact mistake as its top beginner error, on the backend side — the frontend must not manufacture the very input the backend intentionally ignores).

Definition of Done: all tests pass; a teammate has manually walked through confirming from only one side and confirmed the UI correctly shows the transfer as still incomplete, before sign-off — mirroring Backend Plan Task 4.A's own manual sanity-check requirement.

Dependencies for other engineers: the completion event is what makes Task 4.G (Contribution History) and Task 4.H (Impact Dashboard) show new data — no direct code dependency, but the demo flow (Section 41) sequences through this task before those two are meaningfully testable end-to-end.

(Remaining Engineer 4 tasks — 4.A Handover API Service, 4.B Handover Details, 4.D Provider Confirmation / 4.E Seeker Confirmation — folded into 4.C above as the two labeled sides of one component, 4.F Completion State — folded into 4.C step 6, 4.G Contribution History, 4.H Impact Dashboard, 4.I Notifications, 4.J Notification Read/Unread State, 4.K Reports UI, 4.L Report Creation, 4.M Verification UI, 4.N Trust Indicators, 4.O Transfer Error/Conflict Handling — follow the same template, condensed:)

Task	Mission	Design dependency	Key acceptance criterion
4.G — Contribution History	Reverse-chronological <code>ContributionCard</code> list, filterable Given/Received	DESIGN.md §13 ContributionCard, §35 “Contribution History” template	Never shows contact info on a contribution row (handover-scoped, time-limited per Backend Plan §17)
4.H — Impact Dashboard	<code>ImpactCard</code> — personal/org/aggregate variants	DESIGN.md §13 ImpactCard, §4 Warm Sand scoping	Aggregate/Admin variant uses neutral treatment, never Sand, per the Design Decision in §4/§27
4.I — Notifications	Reverse-chronological <code>NotificationItem</code> list	DESIGN.md §13 NotificationItem, §35 “Notifications” template	Blocked on the Contract Gap in Section 07 — confirm the actual read-side endpoint before building; do not invent one
4.J — Notification Read/Unread State	Bold+dot unread vs. regular read styling	DESIGN.md §13 NotificationItem	Unread state communicated via <code>aria-label</code> prefix, never dot color alone
4.K — Reports UI	<code>ReportStatus</code> display, reporter-facing variant	DESIGN.md §13 ReportStatus, §25	Reporters see only their own report's status, never outcome details about another user
4.L — Report Creation	Polymorphic report form (target type + reason)	POST <code>/api/reports</code> (Backend Plan §14)	<code>targetType/targetId</code> validated client-side against the actual entity being reported before submit, matching the backend's own validation intent
4.M — Verification UI	Org verification submission + status display	DESIGN.md §25 Trust & Verification table	Pending status shown only on the org's own dashboard, never to public browsers
4.N — Trust Indicators	<code>VerificationBadge</code> consumed everywhere a name appears	DESIGN.md §13 VerificationBadge	Rejected orgs never show a badge at all inline, per the trust model
4.O — Transfer Error/Conflict Handling	403/404/409 mapping specific to Handover/Report endpoints	Backend Plan §16	Every Transfer-domain error code has a mapped, tested UI treatment

12 — Authentication

Register: `RegisterFormComponent` (Task 1.D) submits to `POST /api/auth/register`; on success, `AuthService` stores the returned `AuthSession` and the app navigates to the Dashboard. Frontend validation (Reactive Forms) checks email format and an 8-character-minimum password (mirroring the exact constant the backend enforces, Backend Plan Task 1.A Step 5) purely for immediate feedback — the backend remains the only trusted enforcement point.

Login: same pattern via `POST /api/auth/login`. A failed login shows one generic message regardless of whether the email exists or the password was wrong, matching the backend's deliberate anti-enumeration behavior (Backend Plan Task 1.A Step 2) — the frontend must not “help” by distinguishing these cases itself even if it could infer one from a 401 vs. some other signal.

Logout: stateless per Backend Plan Task 1.A — the frontend calls `POST /api/auth/logout` (which the backend documents as a no-op returning 200) and, regardless of the response, clears the locally held session and redirects to the public landing page. Logout must never be blocked by a network failure — clearing local session state happens client-side first, the API call is best-effort.

Session restoration: on app bootstrap, `AuthService` checks for a stored token; if present, it calls `GET /api/users/me` to both validate the token is still good and refresh the `User` object (catching the case where the user was suspended since the token was issued, per Backend Plan Task 1.A Step 4 — a token can be structurally valid but the account no longer active). A failed restoration call clears the session silently and treats the user as logged out — no error toast on a routine page-refresh check.

Protected routes: `authGuard` (Task 1.G) blocks navigation and redirects to `/login` with a `returnUrl` query param, restoring the original destination after a successful login.

Role detection: `AuthSession.user.role` drives `roleGuard` and Section 08's role-based navigation (Task 1.H) — read once from the session, never separately fetched per-page.

Unauthorized state (401): `errorInterceptor` (Task 1.I) catches any 401 globally, clears the session, and redirects to Login with the banner text specified verbatim in Backend Plan §16 / Product Brief §22: *“Please log in to continue.”*

Forbidden state (403): shown as a page-level Alert or Dialog per context (Section 17) — the primary defense is that the UI never shows an action the user's role/ownership doesn't support in the first place (Section 13); the 403 handler is a backstop for stale UI state, exactly as Backend Plan §16/Product Brief §22 frame it.

Session expiration: identical to the 401 path above — the interceptor does not distinguish “never logged in” from “token expired,” since both resolve to the same user-facing action (log in again).

13 — Authorization / RBAC

Three roles exactly as Product Brief §14 and Backend Plan §07.2 define them: `user`, `organization` (a verification status on an Organization tied to a `User` via `ownerUserId` — **not** a separate account role, mirrored faithfully from the backend's own clarification in Backend Plan §07.2), and `admin`.

Frontend authorization is a UX/security boundary, not the authoritative security boundary. The backend remains authoritative (Backend Plan §07.2/§17). Every guard, hidden button, and disabled action in this document exists to give users a coherent, honest experience of what they can do — never as the actual enforcement mechanism, which always lives server-side.

Role-aware routes: `roleGuard(['admin'])` on every `/admin/*` route; `orgVerifiedGuard` on the standing-request creation route (Product Brief §14's "Publish Standing Request" row — organization-only, and further gated on `verification.status === 'approved'`, matching Backend Plan §07.2's clarification that this is a business rule living in the service layer, not the generic role check — the frontend mirrors that same two-layer check: role gate, then a separate verified-status gate).

Role-aware navigation: Section 08 Task 1.H, driven by the permission matrix transcribed from Backend Plan §07.2 (reproduced here for frontend reference, values copied verbatim, not reinterpreted):

Capability	User	Organization	Admin
Create Resource	✓	✓ (as <code>providerOrgId</code>)	—
Update/Delete own Resource	✓ (owner)	✓ (owner)	✓ (any)
Create Request	✓	✓ (as <code>requesterOrgId</code>)	—
Publish standing/recurring Request	—	✓ (if verified)	—
Generate/Accept/Reject Match	✓ (party to it)	✓ (party to it)	—
Confirm Handover	✓ (provider or seeker on it)	✓	—
Create Report	✓	✓	✓
Resolve Report	—	—	✓
Approve/Reject Org Verification	—	—	✓
Manage Categories	—	—	✓
Suspend/Reactivate User	—	—	✓
View own Contribution History	✓	✓	✓
View Platform Analytics	—	limited (own org)	✓ (full)

Hidden vs. disabled actions: an action the user's role structurally cannot ever perform (e.g., a `user` seeing "Manage Categories") is hidden entirely, not shown-and-disabled. An action the user's role *can* perform in general but not on this specific item (e.g., an owner-only Edit button on someone else's resource) is also hidden, not disabled-with-a-tooltip — `DESIGN.md` §27's 403-handling note explicitly frames the 403 response as a backstop for "stale UI state," implying the primary UI already never offers the action.

403 handling: see Section 12 above — Alert/Dialog per `DESIGN.md` §12.

Admin-only pages: the entire `/admin/*` route branch, rendered inside `AdminLayoutComponent` (Task 1.N), structurally separate from the user shell per `DESIGN.md` §23's "never a single merged menu" rule.

14 — API Contract Integration

Every endpoint below is copied verbatim from Backend Plan Section 14 — no endpoint name, verb, or path is invented. "Owner" is the frontend engineer whose feature calls it.

Method	Endpoint	Feature	Auth	Role/Owner	Loading state	Success state	Known errors	Owner
POST	/api/auth/register	Register	public	—	button spinner	redirect to Dashboard	400, 409	Eng 1
POST	/api/auth/login	Login	public	—	button spinner	redirect to Dashboard	400, 401, 403	Eng 1
POST	/api/auth/logout	Logout	auth	self	none (best-effort)	redirect to Landing	401 (ignored)	Eng 1
GET	/api/users/me	Session restore, Profile	auth	self	skeleton	—	401	Eng 1
PUT	/api/users/me	Profile edit	auth	self	button spinner	Toast "Profile updated"	400, 401	Eng 1
POST	/api/organizations	Org registration	auth	self – owner	button spinner	redirect to Org Dashboard, "Pending verification" banner	400, 401	Eng 1
GET	/api/organizations/:id	Org profile (public)	public	—	skeleton	—	404	Eng 1
PUT	/api/organizations/:id	Org profile edit	auth	owner/admin	button spinner	Toast	400, 403, 404	Eng 1
GET	/api/admin/users	Admin user list	auth	admin	skeleton table rows	—	403	Eng 1
PUT	/api/admin/users/:id/suspend	Admin suspend	auth	admin	button spinner	Toast + row update	403, 404	Eng 1
GET	/api/categories	Category selector	public	—	skeleton	—	—	Eng 2
POST/PUT/DELETE	/api/categories/:id	Category admin CRUD	auth	admin	button spinner	Toast	400, 403, 404, 409	Eng 2
GET	/api/resources	Browse Resources	public	—	skeleton cards	—	—	Eng 2
POST	/api/resources	Create Resource	auth	self – owner	button spinner	Toast "Resource published" (once status action follows), redirect to detail	400, 401, 403	Eng 2
GET	/api/resources/:id	Resource Detail	public	—	skeleton detail	—	404	Eng 2
PUT	/api/resources/:id	Edit Resource	auth	owner/admin	button spinner	Toast	400, 403, 404, 409	Eng 2
PUT	/api/resources/:id/status	Resource lifecycle action	auth	owner/admin	button spinner	Toast (message varies by action)	403, 404, 409	Eng 2
DELETE	/api/resources/:id	Cancel Resource	auth	owner/admin	Dialog confirm → spinner	Toast	403, 404	Eng 2
GET	/api/requests	Browse Requests	auth	—	skeleton cards	—	401	Eng 3
POST	/api/requests	Create Request	auth	self – owner	button spinner	Toast, redirect	400, 401, 403	Eng 3
PUT	/api/requests/:id	Edit Request	auth	owner/admin	button spinner	Toast	400, 403, 404, 409	Eng 3
PUT	/api/requests/:id/status	Request lifecycle action	auth	owner/admin	button spinner	Toast	403, 404, 409	Eng 3
DELETE	/api/requests/:id	Cancel Request	auth	owner/admin	Dialog confirm → spinner	Toast	403, 404	Eng 3

GET	/api/matches	My Matches	auth	self (party)	skeleton match cards	—	401	Eng 3
POST	/api/matches/:resourceId/generate	Generate Matches	auth	resource owner/admin	skeleton match cards	ranked list renders	403, 404, 409	Eng 3
PUT	/api/matches/:id/accept	Accept Match	auth	party to match	button spinner	Toast "Match accepted" + Lifecycle advances	403, 404, 409	Eng 3
PUT	/api/matches/:id/reject	Reject Match	auth	party to match	button spinner	Toast	403, 404, 409	Eng 3
POST	/api/transactions/:matchId/confirm	Confirm Handover	auth	party to handover	button spinner	indicator dot fills; success state on completion	403, 404, 409	Eng 4
POST	/api/reports	Create Report	auth	any authenticated	button spinner	Toast "Report submitted"	400, 401, 404	Eng 4
GET	/api/reports	Admin Reports queue	auth	admin	skeleton table	—	403	Eng 4
PUT	/api/reports/:id/resolve	Resolve Report	auth	admin	Dialog confirm → spinner	Toast, row update	400, 403, 404	Eng 4
GET	/api/users/me/contributions	Contribution History	auth	self	skeleton list	—	401	Eng 4
GET	/api/admin/analytics	Admin Analytics	auth	admin	skeleton KPI cards	—	403	Eng 4
POST	/api/organizations/:id/verify	Admin org verification decision	auth	admin	Dialog confirm → spinner	Toast, status updates	400, 403, 404	Eng 1

Every list endpoint uses the shared `?page=1&limit=20` convention (Backend Plan §14's closing note) — no frontend service invents a different pagination query shape.

15 — Data Models / TypeScript Types

All shared interfaces live in `core/models/`, one file per entity, and are the literal TypeScript expression of Section 07's contract table. General rules:

- **No `any`**. Every HTTP response is typed; an untyped API response is treated the same as a missing test — not acceptable to merge.
- **IDs** are typed as `string` (MongoDB ObjectIds serialize to strings over JSON) — never `number`.
- **Dates** arrive as ISO strings from the API and are typed `string` at the model level (`createdAt: string`), converted to `Date` only at the point of display via a shared `relative-date.pipe.ts / absolute-date.pipe.ts` (matching DESIGN.md §32's relative-vs-absolute date rule) — never stored as `Date` objects in component state, to avoid timezone-serialization bugs when the value round-trips through a form.
- **Enums** are TypeScript string-literal unions, not `enum` keywords (simpler for beginners to reason about, and matches DESIGN.md §34's naming convention `status: 'pending' | 'verified' | 'rejected'` exactly).
- **Optional fields** use `?` and are genuinely optional in the API contract — never used to paper over an unclear requirement (if a field's optionality is unclear, that's a Cross-Engineer Contract Gap, Section 07, not a guess).

```
// core/models/resource.model.ts
export type ResourceStatus =
  | 'draft' | 'published' | 'available' | 'matched'
  | 'accepted' | 'in_handover' | 'completed' | 'impact_recorded'
  | 'unavailable' | 'expired' | 'cancelled';

export interface Resource {
  id: string;
  title: string;
  category: Category;
  quantity: number;
  description: string;
  location: { area: string; city: string };
  availabilityWindow: { start: string; end: string };
  status: ResourceStatus;
  providerId: string;
  providerOrgId?: string;
  createdAt: string;
}
```

```
// core/models/match.model.ts
export interface MatchScoreBreakdown {
  category: boolean;
  location: boolean;
  quantity: boolean;
  urgency: boolean;
  availability: boolean;
}

export type MatchStatus = 'proposed' | 'accepted' | 'rejected' | 'expired';

export interface Match {
  id: string;
  resourceId: string;
  requestId: string;
  score: number; // integer 0-100 per DESIGN.md §16's whole-number rule
  scoreBreakdown: MatchScoreBreakdown;
  status: MatchStatus;
  providerId: string;
  requesterId: string;
  createdAt: string;
}
```

```
// core/models/api-error.model.ts
export interface ApiError {
  code: string;
  message: string;
  fieldErrors?: Record<string, string>;
}
```

```
// core/models/pagination.model.ts
export interface Paginated<T> {
  data: T[];
  page: number;
  limit: number;
  total: number;
  totalPages: number;
}
```

The full set (`User`, `Organization`, `Category`, `Request`, `Handover`, `Contribution`, `Notification`, `Report`) follows this same pattern, each field traced back to Product Brief §19's Data Model table and Backend Plan §14.

16 — Forms & Validation

Every form in the app follows DESIGN.md §19's anatomy: **Label** → **Control** → **Helper text OR Error message** → **optional Required indicator**.

Covered forms: Register, Login, Profile, Organization (registration + edit), Resource (create + edit), Request (create + edit), Report creation.

The two-layer pipeline, per form:

```
Frontend Reactive Form validators (immediate UX feedback)
↓
API submission
↓
Backend validation (the ONLY trusted layer, Backend Plan §15)
↓
API error response: { success:false, error:{ code:'VALIDATION_ERROR', message, fieldErrors? } }
↓
errorInterceptor normalizes into ApiError
↓
Form component maps ApiError.fieldErrors keys onto the matching FormControl, calling setErrors()
↓
Field-level error renders exactly where a frontend-caught error would have
```

This means every form's error-rendering template code handles frontend-caught and backend-returned errors identically — a `FormControl`'s error state doesn't care which layer caught the problem, which keeps the component simple.

Frontend validators mirror backend rules exactly (never invent a stricter or looser frontend rule than the documented backend one): email format + required; password minimum 8 characters (Backend Plan Task 1.A Step 5's documented constant); resource/request quantity > 0; availability window end > start; category required and must be a real (fetched) category id, never free text; urgency one of the three enum values.

Explicit non-negotiable: frontend validation exists purely for UX — faster feedback, fewer wasted round-trips — and is never treated as sufficient on its own. Every form submission still goes to the backend and still handles a validation-rejection response, even though the frontend “already checked.” This is stated directly in Product Brief §21 and repeated here because it is a common beginner shortcut to skip backend-error handling once frontend validation “already passes.”

17 — Error Handling

Centralized in `errorInterceptor` (Task 1.I). Mapping, copied verbatim from Backend Plan §16 / Product Brief §22 (no alternate wording introduced anywhere in the frontend):

Code	HTTP	UI treatment	Message
VALIDATION_ERROR	400	Inline field error(s) via <code>fieldErrors</code> ; form does not submit; first invalid field receives focus	field-specific, e.g. "Quantity must be greater than 0."
UNAUTHENTICATED	401	Clear session, redirect to Login + banner	"Please log in to continue."
FORBIDDEN	403	Alert or Dialog; action already hidden in the primary UI where the app already knows the user lacks permission	"You don't have permission to do that."
NOT_FOUND	404	Dedicated not-found state on the detail page, link back to Browse	"This listing no longer exists."
CONFLICT	409	Inline banner near the triggering action, using the API's specific returned message	e.g. "This request has already been matched."
INTERNAL_ERROR	500	Toast/banner, generic — never a stack trace or raw error text	"Something went wrong. Please try again."
network failure	—	Persistent banner, automatic retry/backoff; unsaved form input preserved	"Connection lost — retrying..."

API error normalization: `errorInterceptor` catches every `HttpErrorResponse`, extracts the backend's `{ success:false, error:{ code, message, fieldErrors? } }` shape into the shared `ApiError` model (Section 15), and re-throws that normalized shape — no component ever touches a raw `HttpErrorResponse`.

Retry behavior: only the network-failure case auto-retries (with backoff); a 4xx/5xx from a reachable server is never auto-retried (retrying a 403 or a 409 would not change the outcome and could mask a real problem).

Logging: unhandled errors are logged to the browser console in development builds only (stripped in production via Angular's build configuration) — the frontend never ships its own remote error-logging infrastructure for the MVP (out of scope per Section 51's over-engineering guardrails; server-side logging is the backend's documented responsibility, Backend Plan §17).

18 — Loading States

Context	Pattern	Source
Initial page load (route-level data fetch)	Skeleton blocks matching the page's real section layout	DESIGN.md §26
List loading (Resources/Requests/Matches)	3–6 skeleton cards matching real card anatomy	DESIGN.md §26
Detail page loading	Skeleton blocks (header, body, sidebar)	DESIGN.md §26
Form submission	Button loading pattern (label replaced by centered spinner, width preserved, disabled)	DESIGN.md §12 Button
Match generation	Skeleton Match Cards (score numeral skeleton + breakdown row skeletons) — never a fake progress percentage	DESIGN.md §26
Handover confirmation	Button loading pattern	DESIGN.md §12
Route-level navigation	Thin top progress bar only if the route's data fetch exceeds 300ms — no full-page spinner overlay	DESIGN.md §10/§26

The 300ms rule (NFR-006), implemented once, centrally: a shared `LoadingDelayDirective` /utility used by every list/detail component delays showing its skeleton until 300ms have elapsed since the request started — if the response arrives first, no loading UI is ever shown at all (avoids flicker on fast responses, per the exact Design Decision recorded in DESIGN.md §20). This is implemented once in `shared/utls/`, not re-implemented per feature.

Spinners are reserved for buttons and small, bounded regions; skeletons are used for everything content-heavy — this distinction is enforced in code review, not left to individual taste.

19 — Empty States

Every empty state, copied verbatim from `DESIGN.md` §21 (itself matching Product Brief §24's key-screens table), answers **what's empty, why, and what to do next** — never a bare "No data found."

Context	What	Why	Next action	Owner
No resources	"You haven't published any resources yet"	New account or all listings resolved	"Publish a Resource" (Primary)	Eng 2
No requests	"You haven't published any requests yet"	Same pattern	"Publish a Request"	Eng 3
No matches	"No matches yet for your active listings"	Nothing currently qualifies against the scoring threshold	"Browse open requests" / "Check back soon"	Eng 3
No notifications	"You're all caught up"	Nothing unread/recent	none — a positive empty state needs no CTA	Eng 4
No contributions	"No completed transfers yet"	New account or first transfer still in progress	"See your active matches" or "Browse resources"	Eng 4
No search results	"No resources match these filters"	Filters too narrow, or genuinely nothing available	"Clear filters" (Ghost)	Eng 2/3

Every empty-state component is a small, reusable `EmptyStateComponent` (icon + heading + explanatory sentence + optional action slot via content projection) built once by Engineer 1 in `shared/ui/` (Task 1.B's extended scope) and configured per context by the owning engineer — never six separately hand-built empty-state layouts.

20 — Responsive Implementation

Page-by-page behavior, transcribed from `DESIGN.md` Section 8 and applied concretely:

Page	Mobile (<640px)	Tablet (640–1023px)	Desktop (≥1024px)
Landing	Single column, stacked hero	2-column feature highlights	Full hero + 3-column how-it-works
Browse (Resources/Requests)	1-column card stack, filters in a bottom-sheet drawer	2-column grid, filters as a top bar	3–4 column grid, filters as a sidebar or top bar
Resource/Request Details	Stacked sections, full-width	Same, more breathing room	Two-column (content + sidebar summary) where applicable
Create forms	Single column, full-width fields, stacked field pairs (e.g. availability start/end)	Single/two-column mixed	Multi-column where logically grouped
Dashboard	KPI row scrolls horizontally or stacks 1/row; sidebar collapses to bottom tab bar	2 KPIs/row, collapsible sidebar	Full sidebar + 3–4 KPI row
Matches	Stacked Resource-above-Request inside each Match Card	Same	Two-column Resource/Request comparison per card
Handover	Vertical stepper timeline, stacked confirmation panel	Same	Horizontal timeline
Notifications	Full-width list	Same	Same, capped content width
Admin	Tables convert to stacked cards below tablet	Real table	Real table, denser rows

General rule, restated from `DESIGN.md` §8/§30 and enforced without exception: content **stacks** before it **scrolls**, and **scrolls** before it **hides**. Hiding is reserved for genuinely secondary metadata (e.g., “posted 3 days ago”) — category, quantity, and status are never hidden at any breakpoint, matching `DESIGN.md` §5’s truncation rule.

Each engineer verifies their own pages at all three breakpoints as part of that task’s own Definition of Done (Section 39) — responsive behavior is not a separate end-of-project pass.

21 — RTL / LTR

Implementation rules, transcribed from `DESIGN.md` Section 29 and made Angular-concrete:

- **Direction switching:** a single `LanguageService` (Task 1.C) sets `document.documentElement.dir / lang`; every component styles itself using Tailwind's logical-property utilities (`ms-4` not `mL-4`, `pe-2` not `pr-2`) so the same markup works in both directions with zero per-component RTL branching.
- **Icons:** directional icons (arrows, chevrons, send/reply) are wrapped in a small `DirectionalIconComponent` that flips via a CSS transform tied to `[dir]`; non-directional icons (category, status, check/flag/clock) render identically in both directions — no icon asset is duplicated for RTL.
- **Sidebar mirroring:** `AppLayoutComponent / AdminLayoutComponent` (Tasks 1.M/1.N) mirror to the opposite physical side in RTL — a true layout mirror via a CSS logical `inset-inline-start`, not merely flipped text alignment.
- **Forms:** label-above-control layout is direction-agnostic by construction; inline field pairs (availability window start/end) reverse order in RTL via CSS `flex-direction` tied to `[dir]`, preserving "first thing chronologically reads first."
- **Tables:** Admin table column order mirrors in RTL (Task handled inside `shared/ui/table/`, Engineer 1).
- **Pagination/modals:** inherit direction-agnostic layout automatically from logical properties — no special-casing needed.
- **Never mirror blindly:** status badge icons, category icons, and the `MatchScore` checkmarks/crosses stay fixed-orientation in both directions, since they carry no inherent directionality — `DESIGN.md` §29's explicit warning, applied literally in `MatchScoreComponent` (Task 3.K) and the `StatusBadge / UrgencyBadge` components.

Do not duplicate entire screens for Arabic and English — every page is built once, direction-responsive by construction, per the task instructions' explicit Section 24 rule.

22 — Accessibility

Target: **WCAG 2.2 AA**, matching `DESIGN.md` Section 30 and Product Brief NFR-011, applied concretely to Angular:

- **Keyboard navigation:** every interactive element (cards, buttons, filter chips, tabs) is reachable and operable via keyboard alone; card-as-link patterns (`ResourceCard` , `RequestCard` , `MatchCard`) use one focusable wrapping `<a> / <button>` , never nested duplicate focus stops — verified explicitly in Task 2.D/3.A/3.K's testing sections.
- **Focus management:** route changes move focus to the new page's main heading (Task 1.C); modals/dialogs trap focus and return it to the triggering element on close (built into the `Dialog` primitive itself, Task 1.B, per `DESIGN.md` §33's "accessibility contracts ship with the primitive" rule — no consuming feature has to remember to add this).
- **Semantic HTML:** real `<button>` , `<a>` , `<nav>` , `<main>` , `<header>` , `<form>` throughout — a styled `<div (click)>` is treated as a code-review-blocking issue, not a style nitpick.
- **Labels:** every form control has a programmatic `<label>` association; every icon-only button has `aria-label` .
- **Status communication:** never color alone — every status badge, urgency badge, and the `MatchScore` breakdown pairs its color/icon with visible text (`DESIGN.md` §18/§28), enforced as a literal test assertion in the relevant components' specs (Sections 09/10/11).
- **Reduced motion:** `prefers-reduced-motion` honored once, at the token/utility layer (Task 1.A's Tailwind config), never per-component — any transform-based animation (Lifecycle pulse, Skeleton shimmer, Modal scale) is replaced by an instant change or a simple opacity crossfade.
- **Touch targets:** minimum 40×40px on every interactive element, including compact `sm` buttons in touch contexts (Button primitive, Task 1.B, handles this once).
- **Contrast:** every text/background and UI-boundary pairing checked against `DESIGN.md` §4's AA table — `primary-600` is never used for small body text on white (must be `primary-800`), enforced by using semantic Tailwind classes that already encode the correct token, not by ad hoc color picks.

Every task in Sections 08–11 carries its own Accessibility acceptance criterion — accessibility is not a separate end-of-project sweep; per `DESIGN.md` §2's "Accessibility by default" principle, a component that fails its accessibility requirement is not "done," full stop.

23 — State Management

Local component state (Signals): open/closed toggles, active tab, in-progress form state, temporary UI flags (`isMenuOpen` , `selectedTab`). Lives entirely inside the component that owns it; never lifted unless a second component genuinely needs it.

Feature state (small injectable services holding a `signal` / `BehaviorSubject` , scoped to a lazy-loaded feature's providers): the currently-viewed Resource/Request/Match while its detail page and any child components are open; the current filter/search state on a Browse page; in-flight match-generation results. Reset automatically when the user navigates away from the feature (Angular destroys the feature-scoped service instance).

Global state (provided in `root` , exists once for the app's lifetime): `AuthSession` (Task 1.E) — the only piece of state truly global in the functional sense (every guard, every nav item, every ownership check reads it); active language/direction (`LanguageService` , Task 1.C); the global `ToastService` queue (Task 1.K).

Why no NgRx for the MVP: none of the state above requires time-travel debugging, complex derived-state graphs, or state shared across dozens of unrelated features — it requires “one service, one `signal` , read by the handful of components that need it.” Product Brief Section 51 and this plan's own Section 51-equivalent guardrail explicitly warn against unnecessary state-management libraries; introducing NgRx here would mean four beginners learning actions/reducers/effects/selectors to manage what a plain injectable service already solves cleanly. If a genuine cross-cutting state problem emerges during implementation that a plain service cannot solve, it is documented as a new Design/Architecture Decision in this file before any engineer reaches for NgRx — the same “reuse before reinvention” discipline `DESIGN.md §37` applies to visual patterns applies here to state patterns.

24 — Routing Plan

/	Landing (public)
/about	About (public)
/login	Login (public)
/register	Register (public)
/resources	Browse Resources (public)
/resources/:id	Resource Detail (public)
/resources/create	Create Resource (auth)
/resources/:id/edit	Edit Resource (auth, owner/admin)
/resources/mine	My Resources (auth)
/requests	Browse Requests (auth)
/requests/:id	Request Detail (auth)
/requests/create	Create Request (auth)
/requests/:id/edit	Edit Request (auth, owner/admin)
/requests/mine	My Requests (auth)
/matches	My Matches (auth)
/matches/:id	Match Detail (auth, party)
/handovers/:matchId shape exactly	Handover Detail & Confirmation (auth, party) – keyed by matchId, matching Backend Plan §14's confirm endpoint
/dashboard	User Dashboard (auth)
/profile	Profile (auth)
/notifications	Notifications (auth)
/contributions	Contribution History (auth)
/organizations/register	Register Organization (auth)
/organizations/:id	Organization Profile (public)
/organizations/dashboard	Org Dashboard (auth, org owner)
/organizations/verification	Verification status/submission (auth, org owner)
/admin	Admin Dashboard (auth, admin)
/admin/users	Admin Users (auth, admin)
/admin/organizations	Admin Organizations / Verification queue (auth, admin)
/admin/resources	Admin Resources (auth, admin)
/admin/requests	Admin Requests (auth, admin)
/admin/reports	Admin Reports (auth, admin)
/admin/categories	Admin Categories (auth, admin)
/admin/analytics	Admin Analytics (auth, admin)
**	Not Found (public)

Every route above maps to a genuine screen in Product Brief Section 24's Key Screens table and a genuine endpoint in Backend Plan Section 14 — no route requires backend functionality that does not exist. The `/handovers/:matchId` route intentionally keys on `matchId` rather than a handover ID, mirroring Backend Plan Task 4.A's explicit API design choice exactly, so the frontend route param and the API path param are the same value with no lookup translation needed.

25 — Page Inventory

Page	Route	Role	Owner	API dependencies	Components	Priority	Status
Landing	/	public	Eng 1	—	Hero, feature highlights	P0	Planned
Login	/login	public	Eng 1	/api/auth/login	Input, Button	P0	Planned
Register	/register	public	Eng 1	/api/auth/register	Input, Button	P0	Planned
Browse Resources	/resources	public	Eng 2	GET /api/resources , GET /api/categories	ResourceCard, LocationChip, Pagination	P0	Planned
Resource Detail	/resources/:id	public	Eng 2	GET /api/resources/:id	VerificationBadge, StatusBadge	P0	Planned
Create/Edit Resource	/resources/create , /resources/:id/edit	auth	Eng 2	POST/PUT /api/resources*	Form controls, CategorySelector	P0	Planned
My Resources	/resources/mine	auth	Eng 2	GET /api/resources?providerId=me	ResourceCard	P0	Planned
Browse Requests	/requests	auth	Eng 3	GET /api/requests	RequestCard, UrgencyBadge	P0	Planned
Request Detail	/requests/:id	auth	Eng 3	GET /api/requests/:id	UrgencyBadge, StatusBadge	P0	Planned
Create/Edit Request	/requests/create , /requests/:id/edit	auth	Eng 3	POST/PUT /api/requests*	CategorySelector (shared)	P0	Planned
My Requests	/requests/mine	auth	Eng 3	GET /api/requests?requesterId=me	RequestCard	P0	Planned
My Matches	/matches	auth	Eng 3	GET /api/matches	MatchCard, MatchScore	P0	Planned
Match Detail	/matches/:id	auth	Eng 3	PUT /api/matches/:id/accept\ reject	MatchCard, MatchScore	P0	Planned
Handover Detail	/handovers/:matchId	auth	Eng 4	POST /api/transactions/:matchId/confirm	LifecycleTimeline	P0	Planned
User Dashboard	/dashboard	auth	Eng 1 (shell) + all (widgets)	multiple	KPI cards, ImpactCard, NotificationItem preview	P0	Planned
Profile	/profile	auth	Eng 1	GET/PUT /api/users/me	Form controls	P0	Planned
Notifications	/notifications	auth	Eng 4	see §07 Contract Gap	NotificationItem	P1	Blocked on gap
Contribution History	/contributions	auth	Eng 4	GET /api/users/me/contributions	ContributionCard, ImpactCard	P0	Planned
Org Registration	/organizations/register	auth	Eng 1	POST /api/organizations	Form controls	P0	Planned
Org Profile (public)	/organizations/:id	public	Eng 1	GET /api/organizations/:id	OrganizationCard, VerificationBadge	P0	Planned
Org Dashboard	/organizations/dashboard	auth, org	Eng 1	multiple	KPI cards	P0	Planned
Org Verification	/organizations/verification	auth, org	Eng 1/4	verification endpoints (§14)	FileUpload, status badges	P0	Planned
Admin Dashboard	/admin	auth, admin	Eng 4 (integration)	GET /api/admin/analytics	KPI cards	P0	Planned
Admin Users	/admin/users	auth, admin	Eng 1	GET /api/admin/users , suspend	Table, Dialog	P0	Planned
Admin Organizations	/admin/organizations	auth, admin	Eng 1	POST /api/organizations/:id/verify	OrganizationCard (admin-review variant)	P0	Planned
Admin Resources/Requests	/admin/resources , /admin/requests	auth, admin	Eng 2/3	admin list/flag endpoints	Table	P0	Planned
Admin Reports	/admin/reports	auth, admin	Eng 4	GET/PUT /api/reports*	ReportStatus, Table	P0	Planned
Admin Categories	/admin/categories	auth, admin	Eng 2	POST/PUT/DELETE /api/categories/:id	Table, Form controls	P0	Planned
Admin Analytics	/admin/analytics	auth, admin	Eng 4	GET /api/admin/analytics	KPI cards, ImpactCard (neutral variant)	P0	Planned

26 — Shared Component Inventory

Component	Type	Owner	Used by	Key inputs	Key outputs	States	Accessibility	
Button	Primitive	Eng 1	everyone	variant, size, isLoading, isDisabled	(clicked)	default/hover/active/focus/disabled/loading	focus ring never suppressed, aria-busy	I
Input / Textarea / Select / Combobox	Primitive	Eng 1	all forms	label, value, error, disabled	(valueChange)	default/focus/error/disabled	programmatic label, aria-invalid	I
Card	Primitive	Eng 1	every domain card	variant, isInteractive, isSelected	(clicked)	default/hover/selected/loading	single focus stop when interactive	I
Badge	Primitive	Eng 1	status/urgency/count everywhere	variant, icon, label	—	static	never color-only	I
Dialog / Drawer	Primitive	Eng 1	destructive confirmations, filters	isOpen, title	(confirm), (cancel)	closed/open	role=dialog, focus trap built-in	I
Toast	Primitive	Eng 1	global feedback	variant, message	—	entering/visible/exiting	role=status / alert	I
Skeleton / Spinner	Primitive	Eng 1	every loading state	shape	—	shimmer/pulse	aria-busy, aria-hidden on shapes	I
Table / Pagination	Primitive	Eng 1	Admin lists, Browse	columns, data / page, total	(sort), (pageChange)	default/loading/empty	semantic <table>, aria-sort	I
ResourceCard	Domain	Eng 2	Browse Resources, My Resources, MatchCard (compact)	resource, variant	(clicked)	default/hover/unavailable/expired/loading	single focus stop	I
RequestCard	Domain	Eng 3	Browse Requests, My Requests, MatchCard (compact)	request, variant	(clicked)	default/hover/loading	single focus stop	I
MatchCard	Domain	Eng 3	My Matches	match	(accept), (reject)	pending/accepted/rejected	breakdown always visible	I
MatchScore	Domain	Eng 3	MatchCard	score, breakdown	—	static	full sentence aria-label	I
LifecycleTimeline	Domain	Eng 4	Resource/Request/Handover detail	steps, activeStep	—	completed/active/upcoming/failed/cancelled	text label per step, reduced-motion aware	I
LocationChip	Domain	Eng 2	cards, filter bars	label, isInteractive, isSelected	(toggled)	default/hover/selected	real <button>, aria-pressed	I
UrgencyBadge	Domain	Eng 3	RequestCard	level	—	static	icon + text always	I
VerificationBadge	Domain	Eng 1	everywhere a name appears	status	—	verified/pending/(rejected never shown)	aria-label	I
ImpactCard	Domain	Eng 4	Dashboard, Contribution History, Admin Analytics	count, label, variant	—	loading/empty/populated	numeral always has text label	I
ContributionCard	Domain	Eng 4	Contribution History	contribution	—	default/loading	semantic list item	I
OrganizationCard	Domain	Eng 1	Org browse/detail, Admin verification queue	organization, variant	(approve), (reject) (admin-review variant)	pending/verified/rejected	never border-color-only	I
NotificationItem	Domain	Eng 4	Notifications, Dashboard preview	notification	(clicked)	read/unread	aria-label prefix, not dot-only	I
ReportStatus	Domain	Eng 4	Reports UI	report, variant	—	open/reviewed/resolved	standard Badge pattern	I
EmptyState	Utility	Eng 1	every list/empty context	icon, heading, description	content-projected action	—	—	I

27 — Testing Strategy

Unit tests (Jasmine/Karma): services (API wrappers, `AuthService`), pure utilities (`relative-date.pipe.ts`, the `LoadingDelay` utility), validators (custom Reactive Forms validators — `quantity>0`, `date-range`), and individual component logic in isolation (e.g., `MatchScoreComponent`'s rounding and fixed-order guarantees, Section 10's flagship-task tests).

Integration tests (component + `HttpClientTestingModule`, mocked API responses shaped exactly like Section 14's contract): login flow, resource creation, request creation, match generation → accept, handover two-sided confirmation, notification list rendering (once the Section 07 gap resolves).

Route tests: every guard (`authGuard`, `roleGuard`, `orgVerifiedGuard`) tested against both an authorized and an unauthorized simulated session; confirm redirects to the correct destination (`/login` with `returnUrl`, or a 403 page) in each unauthorized case.

UI behavior tests: for every domain component — loading (skeleton renders with correct anatomy), success, error, empty, disabled, and the specific accessibility assertions called out per-task in Sections 08–11 (e.g., `MatchScoreComponent`'s breakdown-always-in-DOM regression test).

End-to-end demo flow test (Cypress or Playwright — team's choice, used consistently, mirroring Backend Plan §03's "team picks one, used consistently" rule): scripted against a seeded backend (Section 28), exercising the exact sequence in Section 41:

```
Register/Login → Create Resource → Publish → Create Request → Publish
→ Generate Match → View Match Score & Breakdown → Accept Match
→ Handover → Provider Confirms → Seeker Confirms → Impact recorded
→ Admin views updated Analytics
```

This single end-to-end test is the frontend's equivalent of the Backend Plan's Section 18 "forced-rollback test" — the one test whose failure means the demo itself would fail, and therefore the one test that must never be skipped before a Phase 8 (Section 30) sign-off.

28 — Mock Data / Development Strategy

Each engineer's feature work can begin before every backend endpoint is fully live, using **typed mock fixtures** that match Section 15's models exactly — never an invented shape. A fixture file lives alongside the feature (`resources.fixtures.ts`) and is swapped for the real `ResourceApiService` call behind a single `environment.useMocks` flag, never hardcoded inline inside a component.

Rule, stated directly: mock data must match the backend contract (Section 14/15) exactly — field names, enum values, and status strings copied verbatim from the Product Brief/Backend Plan, never invented “for now.” A mock `Resource` with a `status: 'live'` (not a real lifecycle value) is a bug waiting to surface once real data arrives, not a harmless placeholder.

When mocks are removed: the moment the corresponding backend endpoint is confirmed live in the shared dev environment (Section 30's Phase gates) — `environment.useMocks` is flipped to `false` for that feature's service, and the fixture file is deleted or moved to `*.spec.ts` - only usage (still valuable for unit tests, no longer used for live development).

29 — Git Workflow

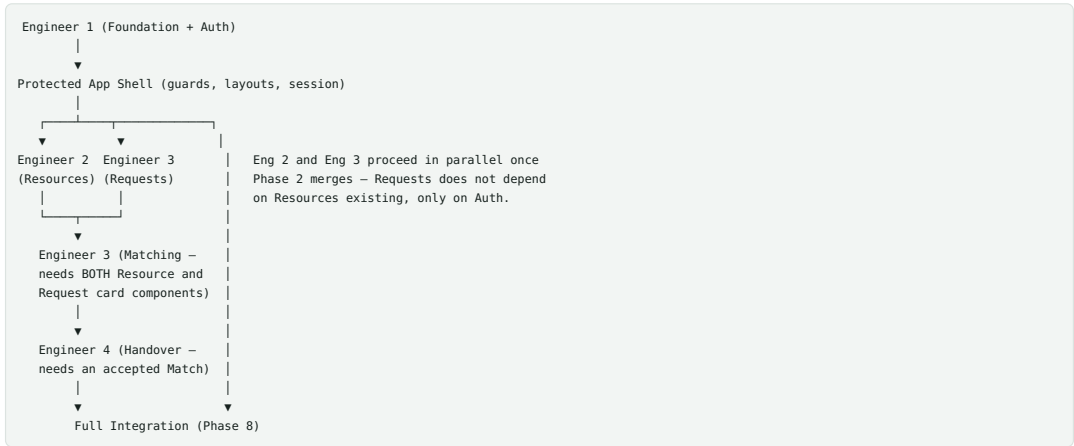
Mirrors Backend Plan Section 20 exactly, applied to the frontend repository:

- **Branches:** `main` (stable, demo-ready), `develop` (integration branch), `feature/ui/<domain>` per engineer — `feature/ui/identity`, `feature/ui/resources`, `feature/ui/matching`, `feature/ui/trust-impact`.
 - **Commit convention:** `ui(<domain>): <what changed>` — e.g. `ui(matching): add MatchScore breakdown grid on mobile`.
 - **PRs required** for every merge to `develop`; small, focused commits — never a single “frontend complete” commit, mirroring the task instructions' explicit warning.
 - **A PR touching another engineer's feature folder or a shared `shared/ui/ / core/` file requires that engineer's explicit approval** — the same shared-file discipline as Backend Plan §20.
 - **Design-system deviations** (any new color/spacing/component not already in `DESIGN.md`) require the PR to include the corresponding Design Decision update to `DESIGN.md` itself, per Section 05 — a PR introducing a new visual pattern without updating `DESIGN.md` is not mergeable.
 - **Cross-engineer contract changes** (Section 07) require sign-off from every consumer listed in that table, not just the owning engineer.
-

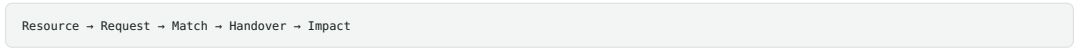
30 — Integration Plan

Phase	Content	Participating engineers	Exit criteria
1	Shared Foundation (Section 08, Tasks 1.A–1.C)	Eng 1	App boots, routes navigable, tokens wired
2	Authentication + Layouts (Tasks 1.D–1.N)	Eng 1	Register/Login work end to end against a real backend; all three layout shells render correctly per role
3	Resources + Requests (Sections 09–10's CRUD tasks)	Eng 2, Eng 3 (parallel)	Full Resource and Request CRUD live; <code>CategorySelectorComponent</code> shared correctly between both domains
4	Matching (Section 10's flagship task)	Eng 3	Generate → display → accept/reject works against a real seeded Resource/Request pair; breakdown always visible
5	Handover (Section 11's flagship task)	Eng 4	Two-sided confirmation demonstrably requires both parties; completion state renders correctly
6	Impact + Notifications (remaining Section 11 tasks)	Eng 4	Contribution History and Impact Dashboard reflect a real completed transfer
7	Admin (Admin routes across all four domains)	all	Every Admin screen functions against its real endpoint
8	Full Integration	all	Section 41's full demo sequence runs start to finish against <code>develop</code> , no manual data patching
9	QA	all, Eng 4 leads	Section 27's full test suite green; manual accessibility and responsive pass on every page
10	Demo Hardening	all	Demo rehearsed at least twice against <code>develop</code> ; no known console errors on any demo-path screen

31 — Dependency Graph



Product-loop dependency (independent of the engineer-ownership graph above, restated from the task instructions):



Category work (Task 2.A/2.B) is on the critical path only insofar as both Resource and Request creation forms need `CategorySelectorComponent` — it must land early in Phase 3, but is not itself gated on anything beyond Phase 1.

32 — Critical Path

The longest dependency chain, and therefore the sequence that determines the earliest possible demo-ready date:

```
Eng 1: Foundation + Design System primitives + Auth + Layouts (Phase 1-2)
→ Eng 2: Resource CRUD (enough to seed a demo resource) (Phase 3)
→ Eng 3: Request CRUD, then Matching (needs both Resource and Request) (Phase 3-4)
→ Eng 4: Handover confirmation (needs an accepted Match) (Phase 5)
→ Eng 4: Contribution/Impact display (needs a completed Handover) (Phase 6)
→ Full Integration (Phase 8)
→ QA (Phase 9)
→ Demo Hardening (Phase 10)
```

Engineer 1's work blocks everyone — no feature route can safely render without auth/guards/layouts existing first, exactly mirroring the Backend Plan's own observation that its Engineer 1 (Identity) blocks the entire backend team. Engineer 2's Resource work is on the critical path only insofar as Matching needs it as an input; it is not itself the bottleneck. The single longest chain is **Engineer 1 → Engineer 3 (via both Resource and Request) → Engineer 4** — this is the sequence that must be protected from slippage above all else.

33 — Risks & Common Failure Points

Risk	Why it happens	Engineer	Prevention	Detection	Recovery
Visual inconsistency across features	Engineers build ad hoc UI instead of reusing primitives	all	DESIGN.md §05 consumption model enforced in code review	Design-review pass before each phase exit	Refactor onto the shared primitive, delete the duplicate
API contract drift	Frontend assumes a field/endpoint shape the backend hasn't actually shipped that way	all	Section 07's Contract Gap protocol; typed models fail to compile on a real mismatch	TypeScript compile errors against real API responses in integration testing	Resolve with the backend engineer, update Section 07, update the model
Auth state bugs (stale session after refresh, race on suspended user)	Session restoration logic not tested against edge cases	Eng 1	Explicit test cases in Task 1.E	Manual refresh-while-logged-in test	Fix AuthService restoration logic
Route guard mistakes (wrong role let through, or wrong redirect)	Guard logic not unit-tested against both authorized/unauthorized cases	Eng 1	Section 27's route-test requirement	Guard unit tests	Fix guard, add missing test case
Duplicate components	An engineer doesn't check shared/ before building	all	Section 05's "primitives/domain components first" rule, PR review checklist	Code review	Refactor onto the existing shared component
Duplicated models	Two engineers each declare their own version of a shared entity	all	Section 07's single-owner contract table	TypeScript type mismatches across features	Consolidate into the owning engineer's core/models/ file
Hardcoded data left in after mocks (Section 28)	environment.useMocks never flipped off for a feature	owning engineer	Phase-gate checklist (Section 30) includes "mocks removed"	Manual review before Phase 8	Flip the flag, verify against real API
Incorrect status handling	UI derives its own status logic instead of rendering the backend's status field/enum	all	Section 07's status-enum contract rule — frontend never computes a status, only displays it	Integration test against real lifecycle transitions	Remove client-side status derivation, render server value directly
Race conditions on double-submit	A button isn't disabled while its request is in flight	all	Button primitive's built-in loading/disabled state (Task 1.B) used consistently	Manual rapid-click test on every write action	Wire the existing loading state correctly
RxJS subscription leaks	A component subscribes in ngOnInit without unsubscribing	all	Prefer the async pipe / Signals (toSignal) over manual .subscribe() wherever possible; documented as the team's default pattern	Angular's dev-mode leak warnings, manual code review	Convert to async pipe or add takeUntilDestroyed()
Bad loading states (spinner instead of skeleton, or vice versa)	Section 18's distinction not followed	owning engineer	Section 18's explicit pattern table	Design review	Swap to the correct pattern
Unhandled errors reaching the user as a raw stack trace	A component catches an error and doesn't route it through errorInterceptor's normalized shape	all	errorInterceptor is the ONLY place errors are formatted (Section 17)	Manual error-injection test per feature	Route the error through the shared pattern
Broken RTL (partial mirroring, physical instead of logical spacing)	An engineer uses mL-4 instead of ms-4	all	Section 21's logical-properties-only rule; a lint rule can catch physical Tailwind spacing classes	Manual RTL toggle test per screen	Replace physical with logical utility classes
Mobile layout failures	A screen only tested at desktop width during development	owning engineer	Section 20's page-by-page responsive table; each task's own Definition of Done	Manual breakpoint walkthrough	Fix the specific breakpoint's layout
Accessibility failures	Keyboard/screen-reader testing skipped under time pressure	owning engineer	Section 22's per-task acceptance criterion	Keyboard-only walkthrough, screen-reader spot-check	Fix per the specific DESIGN.md §30 rule violated
Merge conflicts on shared files	Two engineers edit shared/ui/ or core/ simultaneously	Eng 1 arbitrates	Section 29's shared-file approval rule	PR review	Coordinate sequencing, re-request review
Environment configuration errors (wrong apiBaseUrl in a build)	environment.production.ts not updated before a deploy	Eng 1	Section 37's deployment checklist	Smoke test immediately after deploy	Fix and redeploy

34 — Performance

- **Lazy loading / route-level code splitting:** every feature loads via `loadComponent / loadChildren` (Task 1.C) — the initial bundle contains only the public Landing/Login/Register/Browse-Resources path plus `core/ / shared/ui/` primitives.
 - **Image handling:** resource thumbnails use real aspect ratios inside rounded containers (`DESIGN.md` §3), lazy-loaded via the native `loading="lazy"` attribute — no custom image-optimization pipeline for the MVP (out of scope per Section 51).
 - **Avoiding unnecessary API calls:** debounced search (300ms, Section 09/10), category list cached for the session (Task 2.A) rather than re-fetched per form open.
 - **Pagination:** "Load More" for public Browse screens, numbered pagination for Admin tables (Section 12 Pagination primitive's documented variant split) — never fetching an unbounded list.
 - **Avoiding unnecessary rendering:** `@for` with a `track` expression (Angular's modern control-flow syntax, the equivalent of the old `trackBy`) on every list render (Resource/Request/Match/Notification lists) to prevent full DOM re-creation on every data refresh.
 - **Bundle size awareness:** no component library beyond the project's own primitives (Section 03); Lucide icons imported individually, not as a full icon-font bundle.
 - **Do not over-engineer:** no custom caching layer, no service-worker/PWA infrastructure, no image CDN integration — none of these are required by any FR/NFR, and Section 51's guardrail applies to performance tooling exactly as it does to state management.
-

35 — Security

Frontend-appropriate practices, clearly distinguished from backend security (Backend Plan §17 remains the actual authoritative boundary):

- **Never store sensitive secrets in frontend code** — `environment.ts` holds only `apiBaseUrl`; no API keys, no backend credentials, ever committed.
- **Never trust frontend authorization** — every guard/hidden-button in this document is a UX convenience (Section 13); the backend enforces the real boundary on every request regardless of what the UI shows.
- **Avoid unsafe HTML**: no `[innerHTML]` binding of unsanitized user-generated content anywhere (resource/request descriptions render as plain text via Angular's default interpolation, which auto-escapes — never bypassed with `bypassSecurityTrustHtml` for user content).
- **Protect routes**: every guard from Section 08/13 applied consistently — no protected route reachable via a bookmark/direct-URL bypass.
- **Session expiration**: handled centrally (Section 12) — no feature-specific ad hoc session-expiry handling that might diverge from the global behavior.
- **Never expose backend credentials or raw error detail**: `errorInterceptor` (Section 17) is the single chokepoint that guarantees this.
- **Environment configuration correctness**: production builds always point at the production `apiBaseUrl`, verified as an explicit deployment-checklist item (Section 37).

UX protection vs. actual backend security, stated plainly: every ownership check, role check, and hidden button in this entire document exists to give the user a coherent, non-confusing experience — never as the mechanism that actually prevents unauthorized access. That mechanism is, in every case, the corresponding server-side check documented in Backend Plan Section 07.2/§17.

36 — SEO / Public Experience

Per Product Brief Section 08/24, the public Browse and Landing pages are the primary discoverability surface for anonymous visitors, so:

- **Page titles:** each public route sets a descriptive `<title>` via Angular's `Title` service (e.g., "Browse Resources — Dawwarha").
 - **Metadata:** basic `<meta name="description">` per public route via Angular's `Meta` service — no elaborate structured-data/schema.org investment, which is out of MVP scope.
 - **Semantic HTML:** already required by Section 22's accessibility baseline — this doubles as basic SEO hygiene at no extra cost.
 - **Shareable URLs:** every public resource/request/organization detail page is a real, bookmarkable route (`/resources/:id`) rather than a client-side-only modal state — a shared link opens directly to the right content.
 - **No unnecessary SEO infrastructure:** no server-side rendering (Angular Universal), no sitemap generation pipeline, no prerendering service — none of these are required by any FR/NFR and would meaningfully expand scope beyond the graduation MVP (Section 51 guardrail, applied here too).
-

37 — Deployment

- **Production build:** `ng build --configuration production`, verified locally before every deployment.
 - **Environment variables:** `environment.production.ts` holds `apiBaseUrl` pointing at the deployed backend's origin; Angular's file-replacement mechanism swaps it in automatically for production builds — never a runtime environment variable lookup (Angular builds are static; this is a build-time swap, an important distinction for beginners coming from a Node.js backend mental model).
 - **Development environment:** `environment.ts` points at `http://localhost:<port>` matching the Backend Plan's local dev server.
 - **Staging (if used):** a third `environment.staging.ts` with its own `apiBaseUrl`, built via a separate CLI configuration — optional, added only if the team's deployment target supports a staging slot.
 - **CORS dependency:** the deployed frontend's exact origin must be added to the backend's CORS allow-list (Backend Plan §17) before the two are connected — this is a two-sided coordination step, not a frontend-only task, and is tracked in Section 07 if the origin isn't finalized when frontend deployment prep begins.
 - **Deployment verification checklist:** (1) production build completes with zero errors, (2) the deployed app's network tab shows requests going to the correct `apiBaseUrl`, (3) login works end-to-end against the deployed backend, (4) no console errors on the Landing, Browse, and Dashboard pages, (5) RTL toggle works on the deployed build, (6) a full manual walkthrough of Section 41's demo sequence succeeds against the deployed environment at least once before the live demo.
-

38 — Documentation

Required frontend documentation, so a new team member (or an evaluator) can understand the frontend without asking the original developer:

- **README.md** (repo root): setup steps (`npm install` , `ng serve`), environment configuration, how to run tests, link to this plan and to `DESIGN.md` .
- **Architecture note**: a condensed version of Section 02/04 of this document, kept in `docs/architecture.md` , updated whenever the folder structure genuinely changes (not on every commit).
- **Design system reference**: a pointer to `DESIGN.md` / `DAWWARHA-DESIGN-SYSTEM.md` as the authoritative source — this plan does not duplicate their content, only references section numbers.
- **API integration guide**: `docs/api-integration.md` — how the typed API services are structured, how `errorInterceptor` normalizes errors, how to add a new endpoint following the existing pattern.
- **Environment setup guide**: exact steps to point a local frontend at a local backend (matching ports, CORS notes).
- **Testing guide**: how to run unit/integration/E2E tests, where fixtures (Section 28) live.
- **Contribution guide**: Section 29's Git workflow, condensed into `CONTRIBUTING.md` .
- **Troubleshooting**: a running list of the Section 33 risks and their resolutions, updated as the team actually hits and fixes them during implementation — this becomes the team's own institutional memory by the end of the project.

Every service file (`*-api.service.ts`) carries a header comment: purpose, owning engineer, and which shared models it consumes — mirroring the Backend Plan's own §23 requirement for service-file header comments, so the contract (Section 07) is discoverable from the code itself.

39 — Definition of Done

A frontend feature is not done because “the page works.” It is done only when every item below is true:

- ☐ UI follows `DESIGN.md` / `DAWWARHA-DESIGN-SYSTEM.md` — correct tokens, correct primitive/domain components, no ad hoc styling
 - ☐ Responsive at mobile, tablet, and desktop breakpoints (Section 20), manually verified
 - ☐ RTL and LTR both verified, not just LTR (Section 21)
 - ☐ Loading state implemented per Section 18’s pattern table
 - ☐ Success state is explicit, never silent (Section 34 Do/Don’t in `DESIGN.md`)
 - ☐ Error state implemented per Section 17’s mapping
 - ☐ Empty state implemented per Section 19’s template, where applicable
 - ☐ Validation works, both frontend feedback and backend-error mapping (Section 16)
 - ☐ API contract is respected exactly — no invented field, endpoint, or status value (Section 07/14)
 - ☐ Authorization behavior is correct — action hidden/shown per the real permission matrix (Section 13), understood as UX only, backend remains authoritative
 - ☐ Accessibility requirements are met — keyboard operable, labeled, contrast-checked, screen-reader tested (Section 22)
 - ☐ Tests exist and pass — unit, integration, and (where applicable) route/E2E (Section 27)
 - ☐ No console errors or warnings
 - ☐ No unnecessary duplicated code — reuses existing primitives/domain components (Section 05/26)
 - ☐ Code has been reviewed by at least one other engineer (Section 29)
 - ☐ Integration with the real (not mocked) backend endpoint has been verified (Section 28)
-

40 — Complete Task Checklist

PHASE 0 — Setup - ☐ Repo initialized, `.gitignore`, `environment.ts` skeleton, README skeleton - ☐ `ng new scaffold` + strict TypeScript + Tailwind wired to tokens (Task 1.A)

PHASE 1 — Shared Foundation (Engineer 1) - ☐ TASK 1.A — Angular Application Foundation - ☐ TASK 1.B — Design System Foundation (first-wave primitives) - ☐ TASK 1.C — Routing Architecture - ☐ TASK 1.D — Authentication UI - ☐ TASK 1.E — Auth Session Management - ☐ TASK 1.F — HTTP Interceptor - ☐ TASK 1.G — Route Guards - ☐ TASK 1.H — Role-Based Navigation - ☐ TASK 1.I — Global Error Handling - ☐ TASK 1.J — Global Loading System - ☐ TASK 1.K — Toast/Notification Infrastructure - ☐ TASK 1.L — Public Layout - ☐ TASK 1.M — Authenticated App Layout - ☐ TASK 1.N — Admin Layout - ☐ TASK 1.O — User Profile - ☐ TASK 1.P — Organization UI

PHASE 2 — Design System remaining primitives (Engineer 1, incremental) - ☐ Textarea, Select, Combobox, Checkbox, Radio, Switch, Search, File Upload - ☐ Avatar, Tooltip, Dropdown, Dialog, Drawer, Tabs, Alert, Table, Pagination

PHASE 3 — Supply (Engineer 2) - ☐ TASK 2.A — Category Data Layer & Selector - ☐ TASK 2.B — Category Admin CRUD - ☐ TASK 2.C — Resource API Service (full) - ☐ TASK 2.D — Resource Discovery Page - ☐ TASK 2.E — Resource Search - ☐ TASK 2.F — Resource Filtering - ☐ TASK 2.G — Resource Details - ☐ TASK 2.H — Create Resource - ☐ TASK 2.I — Edit Resource - ☐ TASK 2.J — Resource Lifecycle UI - ☐ TASK 2.K — My Resources - ☐ TASK 2.L — Resource Empty States - ☐ TASK 2.M — Resource Loading States - ☐ TASK 2.N — Resource Error Handling - ☐ TASK 2.O — Resource Responsive UX - ☐ TASK 2.P — Resource Accessibility

PHASE 3 (parallel) — Demand (Engineer 3) - ☐ TASK 3.A — Request API Service & CRUD - ☐ TASK 3.B — Request Discovery - ☐ TASK 3.C — Request Search - ☐ TASK 3.D — Request Filters - ☐ TASK 3.E — Request Details - ☐ TASK 3.F — Create Request - ☐ TASK 3.G — Edit Request - ☐ TASK 3.H — Request Lifecycle UI - ☐ TASK 3.I — My Requests

PHASE 4 — Matching (Engineer 3) - ☐ TASK 3.J — Matching API Integration - ☐ TASK 3.K — Match Results & Score Visualization (flagship) - ☐ TASK 3.L — Match Explanation (folded into 3.K) - ☐ TASK 3.M — (folded into 3.K) - ☐ TASK 3.N — Accept Match - ☐ TASK 3.O — Reject Match - ☐ TASK 3.P — Match Loading/Error/Empty States

PHASE 5 — Handover (Engineer 4) - ☐ TASK 4.A — Handover API Service - ☐ TASK 4.B — Handover Details - ☐ TASK 4.C — Handover Lifecycle UI & Two-Sided Confirmation (flagship) - ☐ TASK 4.D — Provider Confirmation (folded into 4.C) - ☐ TASK 4.E — Seeker Confirmation (folded into 4.C) - ☐ TASK 4.F — Completion State (folded into 4.C)

PHASE 6 — Impact, Trust & Notifications (Engineer 4) - ☐ TASK 4.G — Contribution History - ☐ TASK 4.H — Impact Dashboard - ☐ TASK 4.I — Notifications (blocked on Section 07 Contract Gap) - ☐ TASK 4.J — Notification Read/Unread State - ☐ TASK 4.K — Reports UI - ☐ TASK 4.L — Report Creation - ☐ TASK 4.M — Verification UI - ☐ TASK 4.N — Trust Indicators - ☐ TASK 4.O — Transfer Error/Conflict Handling

PHASE 7 — Admin (all engineers, per domain) - ☐ Admin Dashboard, Admin Users, Admin Organizations, Admin Resources/Requests, Admin Reports, Admin Categories, Admin Analytics

PHASE 8 — Integration - ☐ All `feature/ui/*` branches merged to `develop` in dependency order (Section 31) - ☐ Full end-to-end demo scenario (Section 41) passes - ☐ Cross-domain PRs (Matching ↔ Handover) jointly reviewed

PHASE 9 — Testing & Bug Fixing - ☐ Full test suite (Section 27) green - ☐ Manual accessibility and responsive walkthrough of every page in Section 25

PHASE 10 — Documentation & Demo Prep - ☐ Documentation complete (Section 38) - ☐ Demo rehearsed against `develop` /deployed build at least twice (Section 41)

41 — Graduation Demo Flow

Aligned exactly with the Backend Plan's Section 28 demo flow — the frontend adds no step the backend does not already support, and the backend's every step corresponds to a screen here.

#	Screen	Route	Actor	Action	API call	Expected UI state	What the evaluator should notice
01	Landing	/	—	Narrate the problem	—	Hero + How-it-works	The value proposition is obvious within seconds
02	Login	/login	Provider	Log in with seeded account	POST /api/auth/login	Redirect to Dashboard	Fast, no dead time
03	Create Resource	/resources/create	Provider	Publish a resource	POST /api/resources , then PUT /api/resources/:id/status {publish}	Toast "Resource published"; status badge updates	Guided form, not a flat CRUD form
04	Login (switch)	/login	Seeker/Org	Log in with second seeded account	POST /api/auth/login	Redirect to Dashboard	Role-aware nav differs from the provider's
05	Create Request	/requests/create	Seeker/Org	Publish a request	POST /api/requests , then publish transition	Toast; status badge updates	Same guided-form quality as Resource
06	Matches	/matches	Seeker or Provider	Generate/view matches	POST /api/matches/:resourceId/generate , GET /api/matches	Ranked MatchCard list	The breakdown is visible immediately — this is the moment to pause on
07	Match Detail	/matches/:id	Either party	Accept the match	PUT /api/matches/:id/accept	Toast "Match accepted"; Lifecycle advances	Explainable score language, never "our AI"
08	Handover	/handovers/:matchId	Provider	Confirm handover	POST /api/transactions/:matchId/confirm	Provider's dot fills; "Waiting on your confirmation" on the seeker's side	Two independent indicators, not one bar
09	Handover	/handovers/:matchId	Seeker	Confirm handover	POST /api/transactions/:matchId/confirm	Both dots fill; unmistakable success state	Never a silent status-badge-only change
10	Contribution History	/contributions	Either party	View updated history	GET /api/users/me/contributions	New ContributionCard appears	Factual, non-gamified language
11	Login (switch)	/login	Admin	Log in as admin	POST /api/auth/login	Distinct Admin shell renders	Visually unmistakable operational context
12	Admin Analytics	/admin/analytics	Admin	View analytics	GET /api/admin/analytics	KPI cards reflect the just-completed transfer	Closes the narrative loop the Product Brief describes

Target duration: 5–7 minutes, matching Product Brief Section 37 exactly — every step above corresponds to an endpoint already exercised by the automated test suite (Section 27) before the live demo, mirroring the Backend Plan's own closing guarantee.

42 — Evaluator Experience

The project is an NTI graduation project; the frontend must make the required MEAN-stack capabilities visible during evaluation without adding a single feature that exists only to look good for grading:

NTI Requirement	Where it's visible in this frontend
Authentication	Login/Register forms, session persistence across refresh (Section 12)
Authorization / RBAC	Role-aware navigation (Task 1.H), hidden/disabled actions, distinct Admin shell (Section 13)
CRUD	Resource, Request, Organization, Category full create/read/update/delete surfaces (Sections 09–11)
Validation	Inline field errors on every form, both frontend-caught and backend-returned (Section 16)
Responsive UI	Verified at three breakpoints on every page (Section 20)
API Integration	Every screen backed by a real, typed call to a documented endpoint (Section 14)
Error Handling	Consistent, never-raw-stack-trace error UI across every screen (Section 17)
Reusable Components	The Primitive/Domain component split itself (Sections 05/26) — an evaluator inspecting the codebase sees deliberate architecture, not copy-pasted markup
Clean Architecture	The layered structure in Section 02/04, directly mirroring the backend's own layered discipline
Role-Based UI	Section 13's permission matrix, visibly enforced
Complete Business Workflow	Section 41's demo flow — a resource genuinely becomes a completed, recorded transfer, live

Nothing in this plan is included “for the evaluator” that isn't also genuinely load-bearing for the product — the same discipline the Backend Plan applies to its own scope (no ML matching, no fabricated infrastructure) applies here to the frontend.

43 — Final Architecture Summary

```
Design Tokens (DESIGN.md §4/6/9/10/11)
┆
Primitive Components (shared/ui/ - Section 12/26)
┆
Domain Components (shared/components/ - Section 13/26)
┆
Feature Services (typed API wrappers - Section 14/15)
┆
Pages (route components - Section 24/25)
┆
User Journeys (Section 41's demo sequence)
┆
Product Workflow (Resource → Request → Match → Handover → Impact)
```

Frontend Architecture: Angular 17+, standalone components, Signals for local/UI state, RxJS for HTTP/router streams, functional guards and interceptors, lazy-loaded feature routes — a deliberately simple, typed, modular structure four beginners can hold in their heads at once (Section 03).

Domain Ownership: four engineers, one domain each, matching the backend's own four-way split exactly (Section 06) — Identity & Organizations, Supply, Demand & Matching, Transfer/Trust/Impact.

Component Architecture: strict Primitive vs. Domain separation, both directly derived from DESIGN.md 's own architecture (Section 05) — the frontend does not choose its own component taxonomy, it inherits the design system's.

API Integration: one typed service per backend domain, one normalized error shape, one shared pagination contract — no invented endpoint anywhere in this plan (Section 07/14).

Testing: unit, integration, route, and a single end-to-end demo-flow test that is the frontend's load-bearing proof the product actually works (Section 27).

Deployment: static Angular production build, environment-based API origin, CORS-coordinated with the backend team (Section 37).

Demo Flow: twelve steps, each mapped to a real screen and a real endpoint, closing exactly where the Product Brief's own positioning statement closes — on a completed, recorded transfer (Section 41).

This document, together with [Dawwarha-Product-Brief.pdf](#), the [MongoDB Implementation Plan](#), the [Backend Implementation Plan](#), and [DESIGN.md](#) / [DAWWARHA-DESIGN-SYSTEM.md](#), is the frontend team's complete execution guide from day one until graduation delivery. Any implementation decision not covered here is resolved per Section 05's design-decision hierarchy or Section 07's Cross-Engineer Contract Gap process, and the resolution is recorded back into this file.